

СКАНИРАНО ЗАДАНИЕ

СКАНИРАНО СТАНОВИЩЕ

ПЪРВА ГЛАВА

ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ СИСТЕМИ, ПОДОБНИ НА GIANT

В тази глава са разгледани решения и тенденции, които са пряко свързани с тематиката на дипломната работа: обучение и ускорен инференс на decoder-only езикови модели.

През последните години езиковите модели преминаха от изследователски прототипи към масови потребителски продукти. Появата на ChatGPT и последвалите системи (GPT семейство, Claude, Gemini, open-source LLaMA производни) промени очакванията на потребителите: моделът трябва едновременно да е полезен, контекстно точен и достатъчно бърз за интерактивна работа. Именно това поставя фокуса върху архитектури и decode техники, които подобряват скоростта без рязък компромис в качеството.

В исторически план преходът може да се обобщи така: статистически езикови модели -> рекурентни мрежи -> Transformer (2017) -> мащабирани foundation модели. След Transformer централният въпрос вече не е само “може ли моделът да генерира”, а “може ли да генерира достатъчно качествено и достатъчно бързо за реална употреба”.

1.1 Базов контекст: какво е езиков модел

Езиковият модел е софтуерен модел, който предсказва вероятността за следващ token в текстова последователност. На практика това позволява генериране на нов текст, дописване на изречения и отговори в чат сценарии.

В тази работа се използва decoder-only подход: моделът вижда предишния контекст и предсказва следващия token. По време на обучение параметрите се настройват итеративно, така че грешката между очакван и предсказан изход да намалява. По време на inference параметрите не се променят, а моделът само генерира.

1.2 Термини (кратки дефиниции)

За целите на дипломната документация по-долу са дадени кратки дефиниции на основните термини. След първото им обяснение те се използват в текста и на английски.

Суперскриптните индекси към термините насочват към съответната позиция в раздел “Използвана литература”.

- autoregressive (AR)¹ - режим, при който моделът генерира следващ token на база на предходните токени;
- inference² - процесът на използване на вече обучен модел за генериране, без трениране;

- KV cache² - кеш за Key/Value матриците в attention слоя, който ускорява decode стъпките;
- prefill² - начален forward pass върху prompt-a, който запълва KV cache преди decode;
- decode² - итеративна фаза на генериране на нови токени след prefill;
- sharding³ - разделяне на голям dataset на по-малки shard файлове за по-ефективно четене;
- checkpoint⁴ - запис на състоянието на модела и оптимизатора в конкретна training стъпка;
- resume⁴ - продължаване на training от съществуващ checkpoint вместо start от нула;
- stage⁵ - отделен обучителен етап с конкретни настройки (данни, дължина, loss коефициенти);
- curriculum⁵ - планирана последователност от stages с последователна сложност
- loss function¹ - измерва грешката на модела и определя посоката на оптимизация;
- optimizer⁶ - алгоритъм за обновяване на параметрите по градиент (например Adam/AdamW);
- batch⁹ - група от примери, обработвани заедно в една training стъпка;
- epoch⁹ - едно пълно преминаване през тренировъчния набор данни;
- throughput⁷ - скорост на обработка (например tokens/s);
- latency⁷ - време за единична операция/отговор (например време за decode итерация);
- benchmark² - стандартизирано измерване на производителност при фиксирани условия;
- artifact⁴ - генериран резултат от pipeline/training/inference (лог, checkpoint, графика, отчет);
- LLaMA⁸ - семейство от големи езикови модели (Meta), използвани като архитектурен ориентир;
- RMSNorm¹⁰ - нормализация на активации по RMS стойност;
- RoPE¹¹ - rotary позиционно кодиране за attention;
- SwiGLU¹² - gated feed-forward активация, често използвана в модерни LLM;
- serving² - продукционен режим за подаване на заявки към вече обучен модел.

1.3 Сходни технологии и инженерни практики

1.3.1 LLaMA семейство (Meta)

LLaMA моделите се утвърдиха като де факто стандарт за отворени foundation модели. Основните им предимства са добра scaling стратегия, стабилни архитектурни избори (RMSNorm, RoPE, SwiGLU) и голяма екосистема от производни реализации. За проекта GIANT тези модели са референтна точка за архитектурни и обучителни практики.

1.3.2 SmolLM / компактни open модели

SmolLM профилите са важни като practical baseline за експерименти с ограничен GPU бюджет. В TiDAR частта на проекта се използват точно такива размери (135M и 360M), което позволява многократни сравнения между loss конфигурации и decode варианти в разумно време.

1.3.3 nanoGPT / minGPT

Тези проекти показват минимална, ясно четима реализация на autoregressive Transformer. Те са полезни за концептуално разбиране, но не покриват production-like нуждите от sharding, resume, инфраструктура за отдалечен GPU и систематични benchmark отчети.

1.3.4 vLLM и TensorRT-LLM

vLLM и TensorRT-LLM са системи за ефективно изпълнение на вече обучени езикови модели. Те намаляват закъснението и повишават пропускателната способност чрез оптимизирано управление на KV кеша и GPU изпълнението. В проекта GIANT/TiDAR тези практики се използват като инженерен ориентир при оценката на скоростта и при избора на decode политики.

1.3.5 Защо скоростта е критична за LLM приложения

При интерактивни системи за търсене, чат и асистирано писане скоростта не е “удобство”, а основна част от качеството на продукта. По-високата latency директно намалява вероятността потребителят да продължи взаимодействието.

Класически пример е web search: експериментите на Google (Brutlag, 2009) показват, че увеличение на latency от 100 ms към 400 ms води до измерим спад в броя търсения на потребител, а ефектът се натрупва с времето. Това е пряко свидетелство, че дори “малки” закъснения имат реална продуктова цена.

Отделен клас критични приложения са системите за real-time decision making с AI (оперативни асистенти, monitoring и бърз анализ на събития). В тях латентността влияе пряко върху качеството на решението във времето: при закъснял отговор прозорецът за реакция може вече да е затворен. В такива сценарии ускорение от порядъка 2x-5x е стратегически значимо, защото позволява повече reasoning итерации в същия времеви бюджет.

Именно затова в проекта скоростта се разглежда като първокласна инженерна цел наред с качеството на изхода.

Speed Matters for Google Web Search

Jake Brutlag
Google, Inc.
June 22, 2009

Abstract – Experiments demonstrate that increasing web search latency 100 to 400 ms reduces the daily number of searches per user by 0.2% to 0.6%. Furthermore, users do fewer searches the longer they are exposed. For longer delays, the loss of searches persists for a time even after latency returns to previous levels.

Figure 2: Impact of Post-header Delays Over Time

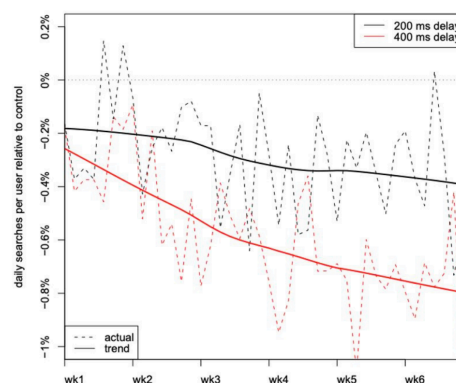


Figure 1: Влияние на latency върху потребителското поведение при търсене

ВТОРА ГЛАВА

ФУНКЦИОНАЛНИ ИЗИСКВАНИЯ И АРГУМЕНТАЦИЯ НА ИЗБОРА НА СРЕДСТВА ЗА РАЗРАБОТКА

Тази глава описва какво трябва да прави системата, защо е избран точно този технологичен стек и как е организиран вътрешният модел на данните и модулите. Целта е читателят да може да проследи логиката от входните данни до крайния training и inference процес, дори без предишен опит с LLM инфраструктура.

2.1 Функционални изисквания

В контекста на тази работа **обучение** (training) означава итеративно нагласяване на параметрите на модела така, че разликата между очакван и предсказан изход да намалява с всяка следваща стъпка. Това става чрез изчисляване на грешка (loss), обратно разпространение на градиентите и обновяване на параметрите от оптимизатор.

2.1.1 Входни източници и ingest pipeline

Системата трябва да поддържа ingest (контролирано зареждане и уеднаквяване) на текстови корпуси от няколко типа източници:

- публични HuggingFace datasets;
- локални JSON/JSONL директории;
- chat-структурирани данни с role/content полета.

Ingest pipeline е последователност от стъпки, която превръща суровия текст във формат, удобен за обучение: нормализация на текст, токенизация, филтриране и запис в стандартизирани shard файлове.

2.1.2 Подготовка на корпус, токенизация и stage-и

Токенизация означава разбиване на текста на по-малки единици (токени), които моделът реално обработва като числа. Вместо да “чете” директно думи, моделът работи с последователности от token ID стойности.

Системата използва **stages** (обучителни етапи). Всеки stage може да има собствена дължина на последователността (seq_len), дял от данните (fraction) и брой епохи (epochs).

Епоха (epoch) е едно пълно преминаване през избраната част от dataset-a за дадения stage.

2.1.3 Конфигурационно управление на експерименти

Експериментът се описва чрез YAML конфигурации. Това е важно за възпроизводимостта: при същите конфигурации и данни резултатът трябва да бъде максимално близък.

Основните нива са:

- глобални настройки в `Global_Config.yml` (пътища, базова среда);
- локални `model/training` настройки в `Config.yml`;
- stage-level настройки за dataset дялове и loss коефициенти;
- CLI overrides за бързи промени без пренаписване на конфигурационните файлове.

2.1.4 Мониторинг и метрики по време на изпълнение

При дълги run-ове е задължително системата да показва междинно състояние, а не само краен резултат.

Затова се изисква:

- логване на loss и помощни метрики по стъпки;
- acceptance метрики (дял приети draft токени) при TiDAR decode;
- запис на checkpoint metadata (стъпка, конфигурация, време);
- автоматично генериране на benchmark отчети.

Метрика е измерима стойност за качество или производителност. Типични примери са loss, accuracy, tokens/s (throughput) и latency.

2.1.5 Управление на хиперпараметри и decode политика

Хиперпараметри са настройки, които не се научават автоматично от модела, а се задават от разработчика: размер на embedding, брой слоеве, размер на batch, learning rate и др.

В тази система управлението обхваща:

- архитектурни параметри (`embedding_size`, `num_heads`, `num_kv_heads`, `num_layers`);
- числен тип на изчисленията (`compute_dtype`, `param_dtype`);
- контекстна дължина и политика за KV cache;
- TiDAR decode параметри (`draft_length`, attention bias, sampling настройки).

2.1.6 Артефакти, checkpoint-и и отчетност

Артефакт е всеки запазен резултат от изпълнението: файл с параметри, лог, графика, отчет.

Системата трябва да съхранява и поддържа:

- dataset shard-ове + manifest + статистики;
- checkpoint-и на параметри + optimizer/dataloader state;
- inference и benchmark артефакти;

- текстови и визуални отчети (.md, .typ, .pdf).

Checkpoint е моментен запис на състоянието на обучението (параметри, optimizer state и metadata) в конкретна стъпка. **Resume** е възстановяване и продължаване на обучението от този запис, вместо нов старт от нула.

2.2 Аргументация за избор на езика и софтуерните средства

2.2.1 Избор на езика за програмиране

Изборът на Python е обусловен от практичността му за ML инженеринг:

- много богата екосистема за data/ML;
- бърз цикъл на прототипиране;
- ясна интеграция със shell автоматизация, Docker и cloud workflow.

2.2.2 Избор на платформа за разработка

Избрана е GPU-ориентирана среда, тъй като обучението и инференсът на LLM са изчислително интензивни и се ускоряват значително чрез паралелната обработка на графичните процесори. Съвременните модели видеокarti разполагат със специализирани изчислителни блокове, които ускоряват матричните умножения - една от основните операции в машинното обучение. Понеже локалната машина не е достатъчна за всички сценарии, работният модел е хибриден:

- локална разработка и подготовка на данни;
- remote GPU изпълнение за тежки run-ове;
- синхронизация чрез Docker + S3 tooling.

Така се постига баланс между цена, скорост и контрол на средата.

2.2.3 Избор на изчислителен фреймуърк

JAX (JIT AutoGrad XLA) е избран заради **JIT** (Just In Time) компилацията и оптимизациите от **XLA** (aXelerated Linear Algebra). На практика Python функцията се трасира и компилира до оптимизиран XLA граф, специализиран за конкретни форми и типове на входа.

Съществена характеристика на JAX е, че следва **чисто функционална парадигма** (pure functional paradigm): изчисленията се описват като функции без скрити странични ефекти, а състоянието се подава явно чрез входове и изходи. Точно тази форма улеснява JIT/XLA трансформациите (компиляция, векторизация, оптимизация на графа) и насочва разработката към optimization-first подход, при който архитектурните и числените решения се проектират с мисъл за ефективно компилируемо изпълнение.

Flax е надстройка над JAX за модулно описание на невронни архитектури и управление на параметри/състояние.

Комбинацията JAX + Flax е ключова за проекта, защото позволява:

- висока производителност;
- ясен модел на train/inference state;
- интеграция с custom KV cache логика и TiDAR маски.

2.2.4 Избор на optimizer/checkpoint инструменти

Optax е използван за optimizer/scheduler логиката, а Orbx - за checkpointing.

Това е важно при дълги обучения, където прекъсване без checkpoint би означавало загуба на часове или дни изчисления.

2.2.5 Избор на средства за deployment и синхронизация

Използват се Docker, Tailscale и S3 съвместими инструменти.

- Docker осигурява еднаква изпълнима среда (еднакви зависимости) между машини;
- Tailscale осигурява защитена частна мрежа (VPN), базирана на WireGuard протокола, за remote достъп;
- S3 workflow позволява надеждна синхронизация на данни и артефакти между локална и remote среда.

2.3 Структура на данните и вътрешен модел на системата

2.3.1 Формат на данните и логика на съхранение

Системата използва файлово-ориентиран модел, а не релационна база данни.

Основният формат за обучителни данни са Arrow shard файлове, придружени от manifest и статистики. Този избор е направен заради бързо последователно четене от data loader-a и добра съвместимост с големи обеми текст.

JSON/TXT формати се използват като входни или междинни формати в ingest етапа, но не и като финален training storage backend.

2.3.2 Stage конфигурация и curriculum модел

Curriculum в този контекст означава планирана последователност от training stages, при която различните етапи имат различни цели и настройки.

```
@dataclass
```

```
class StageConfig:
```

```
    name: str
```

```
    dataset: str
```

```
    seq_len: int
```

```
    epochs: int
```

```
    end_ratio: float
```

Този тип структура позволява прозрачно управление на сложни training сценарии без ръчно пренаписване на кода за всеки нов run.

2.3.3 Конфигурационни домейни на системата

Конфигурацията е разделена на логически домейни:

- `model` - архитектура на мрежата;
- `optimizer` - стратегия за `update`;
- `training` - `batch`, `stages`, `logging`, `checkpointing`;
- `inference` - `decode` режим и `runtime` параметри;
- `tidar` - специфични настройки за TiDAR.

Това разделение улеснява дебъгването и прави експериментите по-проследими.

2.3.4 Схема на dataset записите в shard

Единичният запис в Arrow shard е фиксиран по структура и съдържа:

- `input_ids` - токените;
- `length` - реална дължина на последователността;
- `loss_mask` - позиции, които участват в `loss` изчислението.

`StageDataLoader` преобразува тези полета към моделно удобен `batch` формат (`input/target/mask`).

2.3.5 Източници и ingest адаптери

`StageSourceCfg` описва типа източник и правилата за четене (`plain text`, `JSON`, `chat` формат, `streaming/non-streaming`).

Така един и същ `ingest pipeline` може да обработва различни входни формати без отделни, несъвместими кодови пътища.

2.4 Цялостна архитектура и структурна организация

Архитектурата е разделена на три слоя:

- интерфейсен слой (`CLI` + `YAML` конфигурации);
- управляващи модули (`pipeline`, `training`, `checkpointing`, `inference`);
- математически модел (`Transformer` + `TiDAR` разширения).

2.4.1 Интерфейсен слой (CLI)

Интерфейсният слой е `CLI-first`: потребителят работи чрез `entry point` скриптове и `YAML` конфигурации, което осигурява директен контрол на параметрите, автоматизация чрез скриптове и възпроизводимост на всеки run. Същият интерфейс се използва еднакво при локална работа и при отдалечено изпълнение през `SSH/tmux` върху GPU среда.

В текущата реализация не се използва отделно графично табло от тип TensorBoard. Вместо това системата води активно и структурирано логване на метриките по време на обучението (loss, ppl, acceptance показатели и др.), като тези логове се преобразуват директно във фигури и графики чрез Typst. Примерите за такава визуализация са показани в следващите раздели на документа.

Критичните метрики, които показват дали даден training run трябва да бъде прекратен или коригиран, се извеждат постоянно на терминалния екран по време на изпълнение в tmux. Това осигурява непрекъснат оперативен контрол без необходимост от отделен графичен интерфейс.

2.4.2 Управляващи модули

Управляващите модули са “гръбнакът” на системата:

- data pipeline модул за build на корпус и shard-ове;
- training оркестратор за stages, optimizer стъпки и логване;
- checkpoint мениджър за save/restore/resume;
- inference runtime за prefill/decode и performance отчети.

TiDAR надгражда тази основа със специализирани маски и допълнителни loss механизми, без да нарушава базовия workflow.

2.4.3 Основни entry points

Ключовите скриптове са:

- GIANT/v2/data_pipeline/build_corpus.py
- GIANT/v2/model/Run_training.py
- GIANT/v2/model/Generate_faster.py
- GIANT/v2/model/Chat.py
- TiDAR/model/Run_training.py
- TiDAR/model/inference.py

Така системата запазва ясна граница между базова платформа (GIANT/v2) и изследователски разширения (TiDAR).

2.5 Основен математически модел на трансформер архитектурата

Този раздел представя концептуално и формално принципа на работа на Transformer архитектурата, върху която е изграден GIGANT. Изложението преминава от структурно описание на компонентите към математическа формализация на основните операции.

2.5.1 Теоретична отправна точка: Attention Is All You Need

По-ранните езикови модели обработват последователно с рекурсия, токен по токен. Тази зависимост ограничава паралелизма при обучение и увеличава времето за обработка на дълги входове.

Архитектурата Transformer¹ е въведена в статията **Attention Is All You Need** през 2017 г.¹.

Transformer адресира това ограничение чрез механизма **attention**, при който за всяка позиция се изчислява релевантност спрямо останалите позиции в контекста и се формира претеглена контекстна репрезентация.

Първите широко използвани Transformer системи са encoder-decoder модели, насочени основно към машинен превод. През 2018 г. BERT утвърждава encoder подход с двупосочен контекст при обучение.

Също през 2018 г. започва линията decoder-only модели (GPT), при които се използва причинно-следствена (causal) маска: токен на дадена позиция може да “вижда” само токени от предходни позиции, не може да “вижда в бъдещето”. Тази autoregressive decoder-only постановка се превръща в архитектурна основа на съвременните LLM системи.

Операционната логика на attention може да се обобщи така:

- моделът сравнява текущата позиция с всички допустими позиции според маската;
- оценява колко е релевантна всяка от тях;
- комбинира информацията според тези оценки;
- предава получената смес към следващите слоеве.

Тази конструкция позволява паралелна обработка на цели блокове от токени и е фундаментална за съвременните LLM системи.

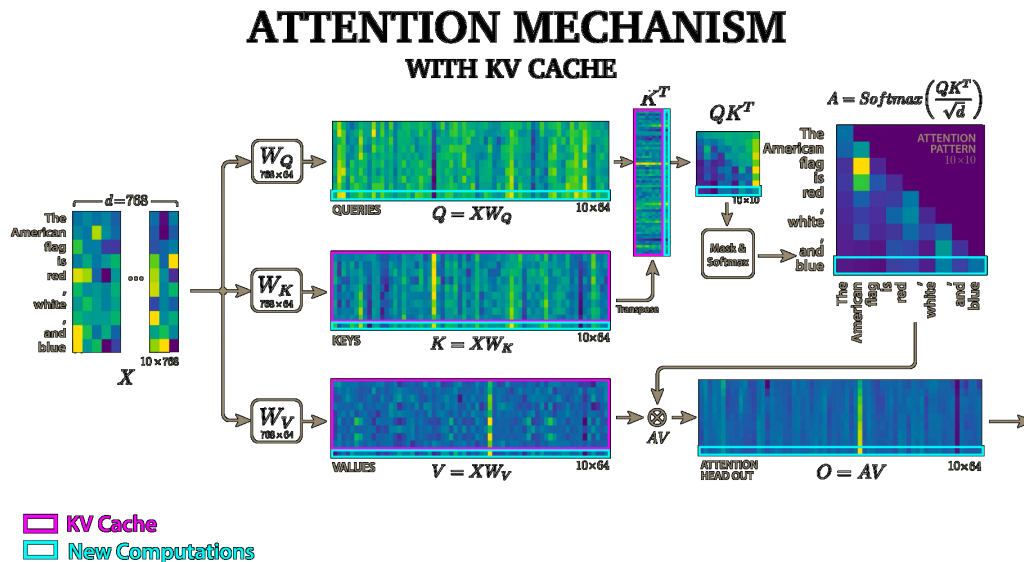


Figure 2: Схема на attention механизма

В математически вид attention операторът е:

$$A(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

където:

- Q (queries) задава какво “търси” текущата позиция;
- K (keys) описва с каква информация разполагат позициите в контекста;
- V (values) е реалното съдържание, което се смесва;
- M е маска (напр. causal маска за decoder режим);
- softmax е функцията за нормализиране на тежестите.

Практически attention минава през три различни представяния, които е важно да се разграничават:

- **scores/logits** - сурови стойности $L = \frac{QK^T}{\sqrt{d_k}} + M$ (още не са вероятности);
- **weights** - резултат след softmax: $W = \text{softmax}(L)$ (вече нормализирани коефициенти);
- **смесени стойности** - финален attention изход: $O = WV$.

Това разграничение е важно, защото logits могат да имат голям мащаб и да бъдат както положителни, така и отрицателни. След softmax те се превръщат във вероятностно разпределение (всички стойности са между 0 и 1 и сумата им е 1), което прави смесването на V стабилно.

Softmax се дефинира като:

$$S(z)_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Така weights се интерпретират като вероятностно разпределение (сборът им е 1), което прави смесването стабилно и интерпретируемо.

Кратък числен пример. Нека за една позиция logits са:

$$z = (2, 1, 0)$$

Тогава:

- $e^z = (e^2, e^1, e^0) \approx (7.39, 2.72, 1.00)$;
- $Z = \sum_j e^{z_j} = 7.39 + 2.72 + 1.00 = 11.11$;
- $S(z) = \frac{1}{Z}(7.39, 2.72, 1.00) \approx (0.665, 0.245, 0.090)$.

Това означава, че първата позиция получава най-голяма тежест, но не “занулява” останалите напълно. Именно така моделът комбинира няколко релевантни места от контекста, вместо да избира само едно.

Делението на $\sqrt{d_k}$ преди softmax е числено важно: без него при големи размерности на векторите dot-product стойностите стават твърде големи, softmax се “насища” и градиентите стават нестабилни.

Класическата форма от оригиналната публикация е:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

В реалните decoder-only модели почти винаги има маска, затова в инженерната форма се използва и M .

В causal режим маската обикновено се задава така: $M_{i,j} = 0$ при $j \leq i$ и $M_{i,j} = -\infty$ при $j > i$.

Тоест позиция i няма право да използва информация от бъдещи позиции $j > i$.

В приложената диаграма (горе вдясно) това е показано с mask анотация от вида $i < j$: червените клетки са забранени (маскирани), а белите са разрешени. Това визуално е същата причинно-следствена идея: “бъдещето” е скрито за текущата позиция.

2.5.2 Multi-head attention: защо са нужни “много глави”

Ако attention е само един, моделът гледа контекста от една “перспектива”. Multi-head attention въвежда няколко паралелни attention подпространства, които могат да уловят различни типове зависимости едновременно:

- синтактични (напр. кой е подлог/сказуемо);
- семантични (напр. кой обект се отнася към кого);

- по-дълги контекстни връзки (препратки между отдалечени токени).

Формално:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

където:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Компактно:

$$H(Q, K, V) = [h_1 \parallel \dots \parallel h_n]W^O, h_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

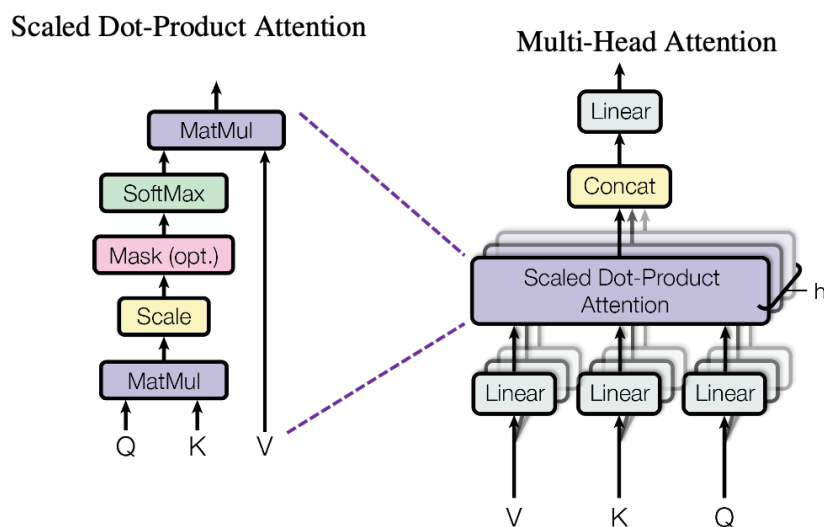


Figure 3: Multi-head attention: блокова схема и формули

2.5.3 От класически Transformer към decoder-only LLM

Оригиналният Transformer е encoder-decoder архитектура. За генеративни езикови модели като GPT се използва **decoder-only** вариант, при който моделът предсказва следващ token от предишните.

Ключовите принципи са:

- **causal masking** - моделът няма право да “гледа в бъдещето”;
- **autoregressive objective** - целта е следващият token;
- еднопосочен поток при генерация.

Причината causal маската да е критична в езиковите модели е, че тя подравнява обучението с реалната употреба. По време на реална генерация бъдещите токени не съществуват, затова и по време на обучение моделът трябва да вижда само ляв контекст.

Това дава две ключови последици:

- по време на **prefill** (когато вече имаме целия prompt) можем да обработим всички позиции паралелно, защото за всяка позиция е ясно кои предишни токени са позволени;

- по време на **decode** работим итеративно, защото новите токени се появяват последователно и всяка следваща стъпка зависи от предишния резултат.

С други думи: causal masking едновременно пази математическата коректност на autoregressive задачата и позволява практичния режим “паралелен prefill + последователен decode”. Точно този шаблон е фундаментален и за TiDAR, където се използват по-сложни структурирани маски, но същата причинно-следствена логика остава валидна.

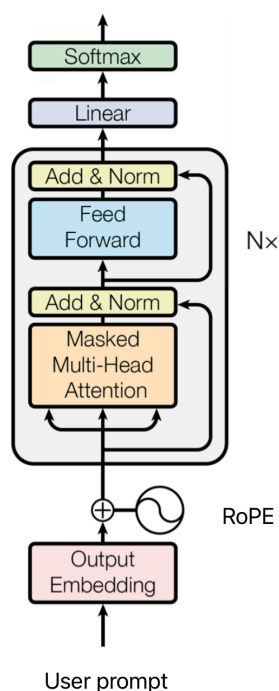
Един decoder блок може да се запише така:

$$x' = x + H(N(x), N(x), N(x))$$

$$y = x' + F(N(x'))$$

където:

- N е нормализация;
- H е multi-head attention;
- F е feed-forward модул;
- добавянето на x и x' са residual връзки за стабилен градиентен поток.



Диаграмата показва пътя от входните токени до вероятностите за следващ token. Входът се преобразува в embedding вектори с позиционна информация, след което минава през поредица от Transformer блокове.

Във всеки блок има causal self-attention (избор на релевантен ляв контекст) и feed-forward модул (локална дообработка на представянията). Residual връзките и нормализацията поддържат стабилно обучение при по-дълбоки модели.

След последния блок следва финална нормализация и проекция към речника. Получават се logits, които след softmax стават вероятности за следващ token.

Практически архитектурата работи в два режима:

- при training позициите в прозореца се обработват паралелно под causal маска;
- при inference се използва “prefill + decode”: prompt-ът се обработва наведнъж, а новите токени се добавят итеративно.

Figure 4: Decoder-only Transformer архитектура

2.5.4 Нормализация, активации и позиционна информация в GIANT

GIANT използва съвременен decoder-only профил:

- RMSNorm¹⁰ вместо класически LayerNorm;
- SwiGLU¹² в feed-forward частта;
- RoPE¹¹ (rotary positional embeddings) за позиционна информация.

RMSNorm¹⁰:

$$R(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} * g$$

RMSNorm държи мащаба на активациите стабилен между слоевете, което подпомага обучението при по-дълбоки модели.

SwiGLU¹² гейтинг:

$$G(x) = s(xW_u) * (xW_v), \quad s(t) = t\sigma(t)$$

SwiGLU работи с два паралелни линейни пътя. Първият път (xW_u) минава през SiLU нелинейност и играе ролята на gate, а вторият (xW_v) носи съдържателния сигнал. Точковото умножение между двата пътя потиска част от каналите и усилва други според контекста.

Това е съществено по-изразително от еднолинейна ReLU MLP схема, защото дава контекстно зависим контрол върху потока на информацията във всеки токенов вектор.

Тук **gate** означава вектор от коефициенти за “пропускане” на информация. След SiLU стойностите в този вектор могат плавно да намалят или увеличат отделни канали. Пътят v е “съдържание” (content branch) - той носи самите признаци, които gate векторът мащабира.

В имплементацията на GIANT (Transformer_block.py) това е реализирано така:

- fc1: проекция от `d_model` към `2 * d_ff`;
- split на две равни части u и v (по `d_ff` неврона всяка);
- `gate = SiLU(u)`;
- `h_ffn = gate * v` (елементно умножение);
- fc2: проекция обратно от `d_ff` към `d_model`.

Числени примери по реални конфигурации:

- TinyStories/TiDAR базов профил (`d_model=384`, `d_ff=1024`): fc1 дава 2048 канала, split до $u = 1024$ и $v = 1024$, после fc2 връща към 384;
- GIANT 135M профил (`d_model=576`, `d_ff=1536`): `fc1 = 3072`, split `1536 + 1536`, после `fc2 = 576`;

- GIANT 360M профил ($d_{\text{model}}=960$, $d_{\text{ff}}=2560$): $\text{fc1} = 5120$, split $2560 + 2560$, после $\text{fc2} = 960$.

По този начин всяка токена позиция получава по-богата и по-контекстна нелинейна трансформация, без да се променя външният размер на блока (d_{model}).

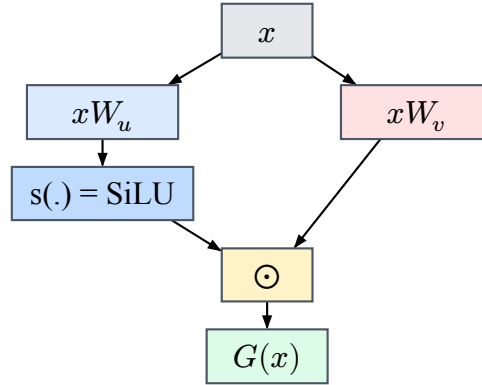


Figure 5: SwiGLU: два линейни пътя + gate + точково умножение

RoPE¹¹ (Rotary Positional Embedding) кодира позицията чрез ротация на компонентите в Q и K вектори. Идеята е всяка двойка координати да се върти с ъгъл, който зависи от позицията. Така позиционната информация се вгражда директно в attention скаларните произведения.

За двойка компоненти

$$(u_{2i}, u_{2i+1})$$

трансформацията е:

$$\begin{pmatrix} u'_{2i} & u'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{pmatrix} \begin{pmatrix} u_{2i} & u_{2i+1} \end{pmatrix}$$

Това дава две практически ползи:

- относителните разстояния между позиции се отразяват естествено в QK^T ;
- моделът обикновено е по-стабилен при по-дълги контексти спрямо по-стари абсолютни позиционни схеми.

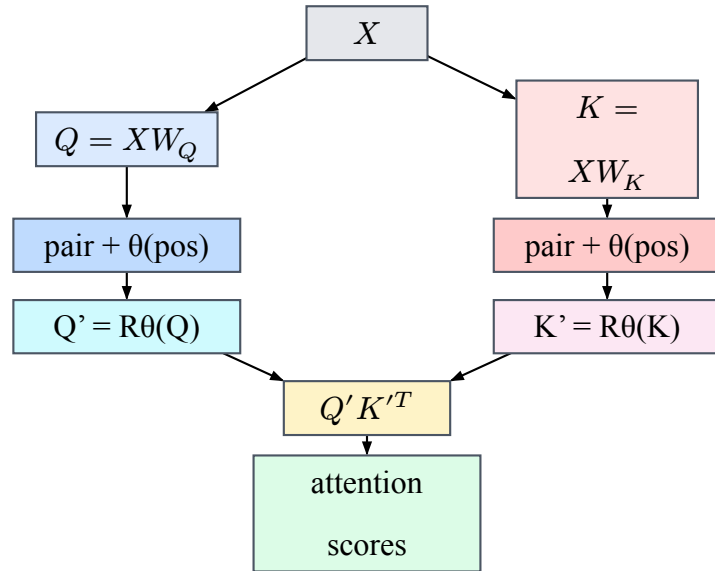


Figure 6: RoPE: позиционно-зависима ротация в Q/K преди attention score

2.5.5 Обучителна цел (AR) и връзка с TiDAR

Базовата обучителна цел на GIANT е autoregressive cross-entropy:

$$L_1 = -\frac{1}{N} \sum_{b,t} m_{b,t} \log p_{\theta}(x_{b,t+1} | x_{b,\leq t})$$

За всяка позиция t в последователността моделът връща вектор от logits $z_{b,t} \in \mathbb{R}^{|V|}$, където $|V|$ е размерът на речника. След softmax получаваме вероятности:

$$p_{b,t} = \text{softmax}(z_{b,t})$$

Целевият токен на тази позиция е цяло число (integer label) $y_{b,t} \in \{0, 1, \dots, |V| - 1\}$. Локалната cross-entropy за позицията е:

$$l_{b,t} = -\log p_{b,t}[y_{b,t}]$$

Маската $m_{b,t}$ указва дали позицията участва в загубата (например $m = 0$ за padding или за позиции извън целевия chat role, и $m = 1$ за валидни обучителни позиции). Общата загуба усреднява само валидните позиции:

$$L_1 = \frac{\sum_{b,t} m_{b,t} l_{b,t}}{\sum_{b,t} m_{b,t}}$$

Така функцията наказва модела силно, когато вероятността за правилния токен е ниска, и слабо, когато е висока. При оптимизацията градиентите повишават логита на правилния клас и понижават част от конкурентните класове, което постепенно подобрява next-token предсказването.

Числен пример за една позиция:

- контекст: The capital of France is;

- целеви токен: `Paris` с integer label `y = id("Paris")`;
- ако моделът дава $p(y) = 0.62$, тогава $l = -\ln(0.62) \approx 0.478$;
- ако моделът дава $p(y) = 0.05$, тогава $l = -\ln(0.05) \approx 2.996$.

Разликата показва защо cross-entropy е подходяща за езиково моделиране: системата получава много по-голям сигнал за корекция, когато правилният токен е оценен с ниска вероятност.

Концептуална визуализация на AR next-token предсказване

Контекст: The capital of France is Целеви етикет: `y = id("Paris")`

Следващ токен (вероятности след softmax):

Paris	: #####	0.62	<- целев клас
London	: #####	0.21	
Lyon	: ##	0.04	
other	: #####	0.13	

Listing 1: Пример за разпределение върху речника при AR обучение (по идея на визуализациите за next-token prediction)

Примерен изход от GIANT v2 checkpoint, трениран върху корпус със силно Wikipedia присъствие (заедно с други източници), е:

User: What is the capital of France?

Assistant: The capital of France is sometimes called Paris.<EOS>

- размер на модела: приблизително 101 милиона параметъра;
- обем на обучението: приблизително 2 милиарда токена.

За мащабно сравнение:

- Llama 3 70B - trained on 15T tokens;
- DeepSeek R1 671B - trained on 14.8T tokens.

TiDAR стъпва директно върху тази основа. Тоест първо се учи стабилен autoregressive модел, а след това се добавят TiDAR разширенията (masking логика и допълнителни loss термини) за по-ефективен decode.

2.6 Спекулативно декодиране (въведение преди TiDAR)

Преди представянето на TiDAR е важно да се въведе идеята за speculative decoding, защото именно тя е базата, върху която стъпва по-късното Anchor-TiDAR разширение.

Класическото autoregressive декодиране генерира по един токен на стъпка. Това е коректно, но често не използва оптимално паралелните възможности на GPU при inference.

Спекулативното декодиране цели да увеличи throughput, като предлага предварителни токени (draft), които после се верифицират и частично или изцяло се приемат. Така за една итерация може да се валидират няколко бъдещи позиции вместо само една.

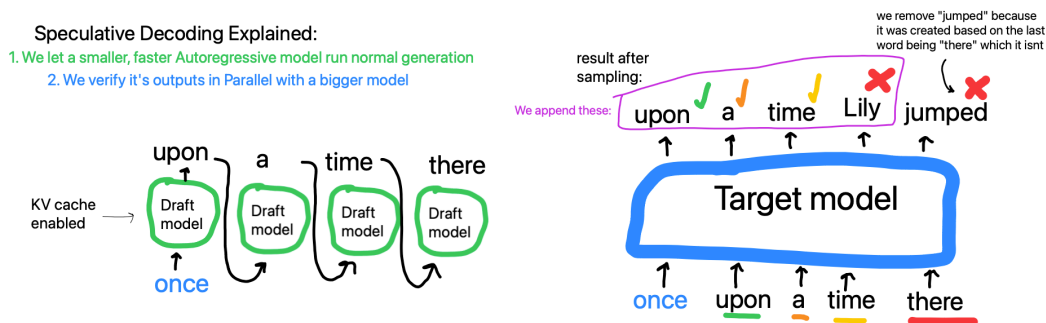


Figure 7: Класически speculative decoding: draft + verify схема

Основната идея е:

- draft е бърза хипотеза за следващите токени;
- verify проверява хипотезата спрямо по-точен/референтен изход;
- приемането на префикс от draft-а води до повече от +1 токен прогрес на итерация.

Тази схема е важна за проекта, защото:

- дава теоретична рамка за ускорение на генерацията;
- естествено се комбинира с KV cache;
- подготвя прехода към TiDAR, където verify и predraft логиката се реализират в един специализиран forward pass.

Важно разграничение: класическите speculative decoding методи обикновено използват втори, по-малък draft модел (например по-малка Llama спрямо по-голям verify модел, напр. 3B срещу 70B) или добавят допълнителни архитектурни компоненти върху базовия модел (например допълнителни глави/слоеве, както при EAGLE).

2.6.1 Защо single-user decode не използва добре GPU

При decode с един потребител effective batch-ът е малък (ограничена паралелизация) и GPU изпълнението често е ограничено от скоростта на паметта, а не от аритметичната пропускателност. Хардуерният профил в този режим може да се обобщи така:

- едно умножение/събиране в изчислително ядро се изпълнява за много малко време (няколко такта);
- четене на данни от глобална HBM/GDDR памет струва значително повече (десетки до стотици тактове);

- ако за всеки малък изчислителен фрагмент постоянно се чака ново четене от памет, ядрата остават частично неактивни.

Данните, които се четат от паметта, са параметри и активации, върху които после се правят изчислителни операции. В decode режим това означава, че за всяка нова стъпка трябва отново да се зареждат части от attention/MLP параметрите и текущите работни тензори.

Затова целта е с едно прочитане на параметри да се свърши повече полезна работа - например повече умножения върху повече токени в рамките на една итерация. Така:

- се вдига arithmetic intensity (повече операции на прочетен байт);
- се увеличава заетостта на Tensor Core блоковете;
- се намалява относителната цена на memory трафика.

В този контекст speculative/TiDAR подходите са приложими, защото вместо +1 токен полезен прогрес на итерация при благоприятен случай се валидират няколко позиции, което амортизира memory достъпите по-ефективно.

В проекта се използва и FlashAttention (cuDNN backend), което намалява IO overhead в attention стъпката и допълнително подпомага този подход.

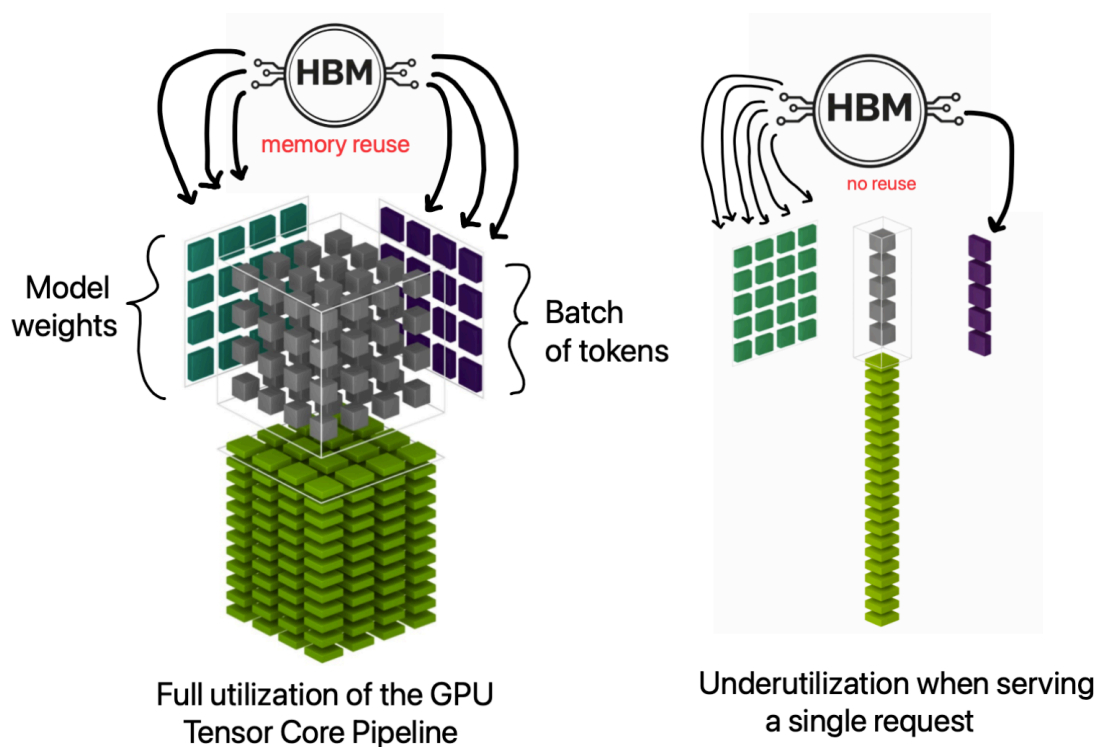


Figure 8: Tensor Core utilization при memory-bound single-user decode

Тази мотивация е важна за разбирането на TiDAR и феномена “Free Token Slots”, където допълнителната паралелна работа в една decode стъпка се използва по-ефективно.

ТРЕТА ГЛАВА

РЕАЛИЗАЦИЯ НА ПРИЛОЖЕНИЕТО

3.1 Реализация на потребителски интерфейс

3.1.1 Основни входни точки (CLI)

Системата е CLI-first и основният интерфейс е набор от entry points:

- `GIANT/v2/data_pipeline/build_corpus.py` - build и шардване на корпус;
- `GIANT/v2/model/Run_training.py` - обучение по stage конфигурации;
- `GIANT/v2/model/Evaluate.py` - офлайн оценка на checkpoint;
- `GIANT/v2/model/Generate_faster.py` - ускорен decode и benchmark;
- `GIANT/v2/model/Chat.py` - интерактивен чат inference;
- `TiDAR/model/Run_training.py` - TiDAR обучение и логване;
- `TiDAR/model/inference.py` - TiDAR inference и сравнения.

Тези скриптове формират основния интерфейс на системата и покриват пълния цикъл: data prep, обучение, оценка и инференс.

3.1.2 Наблюдение и визуализация на резултати

Визуализацията се реализира чрез логове и експериментални отчети:

- training логове (loss, ppl, accept, greedy_accept);
- checkpoint metadata (train_loss, val_loss, val_ppl);
- Typst отчети с throughput и policy сравнения.

Този подход е избран целенасочено за terminal/vim workflow и за лесно наблюдение през SSH.

3.1.3 Управление на stage прогреса

Управлението на прогреса е реализирано чрез stage curriculum. Изпълнението преминава през последователни stages с различен dataset и контекстна дължина, като за всеки етап се съхранява възстановимо състояние.

3.1.4 Управление на параметри

Управлението на параметрите е изнесено в YAML конфигурации и CLI flags. Това позволява силно контролирани експерименти с минимални кодови промени.

3.1.5 Добавяне на входни корпуси

Pipeline-ът поддържа HuggingFace и локални JSON/JSONL входове, chat formatting и loss-mask логика.

3.1.6 Експорт на артефакти

Експорт на артефакти към файловата система/S3:

- Arrow shards;
- manifests/statistics;
- checkpoints;
- benchmark logs;
- Typst/PDF отчети.

3.1.7 Примерен training run в терминал

Следният пример показва типично стартиране на обучение през CLI интерфейса:

```
python GIANT/v2/model/Run_training.py \  
  --config GIANT/v2/model/Config.yml \  
  --global_config GIANT/v2/Global_Config.yml \  
  --checkpoint_dir giant-data/GIANT/v2/checkpoints \  
  --checkpoint_every 500 \  
  --scan_chunk 8
```

Пример за възстановяване на последната налична checkpoint точка:

```
python GIANT/v2/model/Run_training.py \  
  --config GIANT/v2/model/Config.yml \  
  --global_config GIANT/v2/Global_Config.yml \  
  --checkpoint_dir giant-data/GIANT/v2/checkpoints \  
  --resume latest
```

Примерен терминален изход при старт от нула (съкратен):

```
[loader] loading shards from /proj/giant-data/GIANT/v2/processed/stage_base  
...  
[loader] stage=base rows=12800000/12800000 ctx=1024 steps_per_epoch=6250  
  
→ Stage base: seq_len=1024 epochs=1 steps=6250 (resume at 0)  
step    100/10449   | stage base           loss 4.8921 ppl 133.21 (18.4s)  
step    200/10449   | stage base           loss 4.6017 ppl 99.66 (17.9s)  
...  
step    500/10449   | stage base           loss 4.2143 ppl 67.64 (17.2s)  
[metadata] Wrote train_loss=4.214300 to checkpoint .../params/step_0000500.npz  
📁 checkpoint → .../params/step_0000500.npz
```


Примерен терминален изход при `--resume latest` (примерно от стъпка 5432):

```
↳ Resumed from mini checkpoint at step 5432
▶ Restored dataloader state at stage 0 step 5432

→ Stage base: seq_len=1024 epochs=1 steps=6250 (resume at 5432)
step    5500/10449   | stage base                loss 3.9981 ppl 54.50 (16.9s)
...
→ Stage wiki: seq_len=1024 epochs=1 steps=4199 (resume at 0)
step    6400/10449   | stage wiki                 loss 3.7129 ppl 40.97 (16.8s)
...
✓ Training complete. Final checkpoint: .../params/step_0010449.npz
```

Примерен multi-stage TiDAR training run с различни loss конфигурации по етапи:

```
python TiDAR/model/Run_training.py \
  --config TiDAR/model/training_configs/Greedy_exp_135m.yml \
  --global_config TiDAR/Global_Config.yml \
  --checkpoint_dir giant-data/TiDAR/checkpoints \
  --checkpoint_every 400 \
  --resume latest
```

```
[loader] loading shards from /proj/giant-data/TiDAR/processed/stage_base
...
[loader] stage=base rows=6200000/6200000 ctx=1024 steps_per_epoch=3027
[loader] loading shards from /proj/giant-data/TiDAR/processed/stage_chat
...
[loader] stage=chat rows=1800000/1800000 ctx=1024 steps_per_epoch=878

→ Stage base: seq_len=1024 epochs=1 steps=3027 (resume at 0)
step      1/3905    | stage base                loss 5.1023 ar 4.8122 diff
5.2761 accept 0.117 greedy_acc 0.081 (19.2s)
step     200/3905   | stage base                loss 4.4311 ar 4.1926 diff
4.5214 accept 0.223 greedy_acc 0.147 hard 3.8872 (17.4s)
...

→ Stage align_kl: seq_len=1024 epochs=1 steps=1000 (resume at 0)
step     3200/3905  | stage align_kl            loss 3.9027 ar 3.7710 diff
```

```

3.9488 accept 0.356 greedy_acc 0.244 kl_fwd 0.1821 kl_rev 0.0964 distill 0.1418
topk 0.0889 (16.9s)
step    3600/3905    | stage align_kl            loss 3.7814 ar 3.6562 diff
3.8241 accept 0.391 greedy_acc 0.269 kl_fwd 0.1499 kl_rev 0.0792 distill 0.1184
topk 0.0711 (16.6s)
...

→ Stage chat: seq_len=1024 epochs=1 steps=878 (resume at 0)
step    3900/3905    | stage chat            loss 3.7442 ar 3.6214 diff
3.7811 accept 0.405 greedy_acc 0.281 hard 3.1027 distill 0.1021 (16.4s)

[metadata] Wrote train_loss=3.744200 to checkpoint ../params/step_0003905.npz
✓ Training complete. Final checkpoint: ../params/step_0003905.npz

```

3.2 Реализация на основните системни модули

3.2.1 ConfigLoaderManager

Конфигурациите се зареждат чрез merge на глобален и локален YAML, след което се резолват относителните пътища спрямо data_root. Това осигурява еднакъв кодов път за локално и remote изпълнение.

```

def load_configs(config_path=None, global_config_path=None):
    local_cfg = OmegaConf.load(config_path)
    global_cfg = OmegaConf.load(global_config_path)
    cfg = OmegaConf.merge(global_cfg, local_cfg)

    # Унифициране на пътищата спрямо data_root
    cfg.paths.processed_data_root = resolve_path(cfg.paths.processed_data_root,
    cfg.paths.data_root)
    cfg.paths.logs_root = resolve_path(cfg.paths.logs_root, cfg.paths.data_root)
    return cfg

```

3.2.2 TrainLoopOrchestrator

Run_training.py реализира scan-базиран training loop с gradient accumulation, finite checks и checkpoint политика.

3.2.2.1 Основен JIT training loop (_run_chunk + lax.scan)

Сърцето на изпълнението е _run_chunk(...), където batch-овете се обработват със lax.scan, пресмятат се loss/grad стойности и се правят проверки за non-finite стъпки.

`lax.scan` е JAX примитив за “компилиран `for`-цикъл”: вместо Python да `dispatch`-ва всяка стъпка поотделно, целият цикъл се представя като една JIT-оптимизирана граф операция с `carry` състояние (например `params`, `opt_state`, `accum_grads`, `accum_count`) и последователност от изходи (`losses`). Това намалява `host overhead` и дава по-стабилна производителност при дълги `training run`-ове.

3.2.2.2 Обработка на сигнали и контролирано спиране

Скриптът обработва `SIGINT/SIGTERM` и спира контролирано след текущия `chunk`, за да не се загуби междинно състояние. Практически се поддържат два слоя на устойчивост:

- големи `checkpoint`-и (`params` + `optimizer` + `dataloader state`), които са “стабилните” запазвания и се пазят;
- мини `checkpoint`-и, които се записват периодично за бързо възстановяване при внезапно прекъсване (`preemption`, `срив`, `рестарт`).

Така при неочаквано прекъсване може да се продължи от почти последната стъпка, а при нормален `stop/етапен преход` остават и пълните `checkpoint`-и за дългосрочно съхранение и проследимост. В практиката `training run`-овете се поддържат през `tmux` (включително в `CI/CD workflow`), което позволява стабилна сесия, лесно повторно закачане към машината и безопасно дълго изпълнение.

3.2.2.3 Gradient accumulation и update политика

Обучението поддържа `gradient_accumulation`; `update` се прилага само при достигане на зададения `accumulation` праг, а остатъчните градиенти се доизчистват при край на `stage`.

```
# Натрупване на градиенти
accum_grads = tree_map(lambda a, g: a + g, accum_grads, grads)
accum_count = accum_count + 1

# Update само когато достигнем gradient_accumulation
should_update = accum_count == grad_accum
if should_update:
    grads_scaled = tree_map(lambda g: g / grad_accum, accum_grads)
    updates, opt_state = optimizer.update(grads_scaled, opt_state, params)
    params = optax.apply_updates(params, updates)
    accum_grads = tree_map(jnp.zeros_like, accum_grads)
    accum_count = 0
```

На практика това е логика за “мини `batch`-ове” на ниво `update`. Вместо целият глобален `batch` да се побере наведнъж в паметта, той се разделя на няколко по-малки стъпки (`micro-batches`), които

GPU може да обработи с наличния VRAM. Градиентите от тези micro-batches се натрупват и optimizer update се прави накрая, така че се получава ефект на по-голям глобален batch без изискване за голяма еднократна памет.

Този подход е особено важен при по-слаби GPU конфигурации и може да се разшири директно към multi-GPU обучение, където освен локалното accumulation се добавя и синхронизация/редукция между устройствата преди финалния update.

3.2.2.4 Периодично логване на метрики

На всеки log_every стъпки се записват global_step, stage, loss и допълнителни показатели (напр. ppl, accept, greedy_acc, KL/hard/distill/top-k при TiDAR).

Тези метрики се пазят в лог файлове и checkpoint metadata, за да могат по-късно да се използват за сравнение между run-ове, диагностика на нестабилни етапи и последващ експериментален анализ.

3.2.2.5 Resume от checkpoint и dataloader state

Възстановяването зарежда параметри, optimizer state и dataloader прогрес (stage_index, stage_step_total, stage_states), така че обучението да продължи от последната консистентна точка.

```
@jax.jit
```

```
def _run_chunk(params, opt_state, batch_chunk, start_step, accum_grads,
               accum_count):
    # Една компилирана единица работа: loss/grad + accumulation + update
    (params, opt_state, _, accum_grads, accum_count), losses = jax.lax.scan(
        body, (params, opt_state, start_step, accum_grads, accum_count),
        batch_chunk
    )
    return params, opt_state, accum_grads, accum_count, losses
```

3.2.3 DataLoaderManager

ShardedArrowDataset + StageDataLoader реализират shard-aware четене, shuffle, batching и state restore.

```
dataset = ShardedArrowDataset(data_path)
loader = StageDataLoader(
    dataset,
    batch_size=batch_size,
    seq_len=stage.seq_len,
    shuffle=stage.shuffle,
```

```

        seed=seed,
        pad_token_id=pad_token_id,
    )

```

3.2.4 CheckpointManager

checkpoint_manager.py капсулира запис/зареждане на параметри, optimizer state и metadata.

```

ckpt_file = save_ckpt(params, global_step, params_dir, train_loss=last_loss)
save_opt_state(opt_state, global_step, training_states_dir)
save_data_loader_state(state_path, loader_state)

```

При resume

```

params, global_step = load_ckpt(ckpt_path)
opt_bytes = load_opt_state(global_step, training_states_dir)

```

3.2.4.1 Метод SaveParameters

Запис на .npz checkpoint с атомарно tmp -> rename поведение.

3.2.4.2 Метод SaveOptimizerState

Паралелен запис на optimizer буфери (msgpack) за коректен resume.

3.2.4.3 Метод RestoreLatest

Намиране на последната достъпна checkpoint стъпка и зареждане на съответното състояние.

3.2.4.4 Метод SetMetadata

Запис на метаданни за quality показатели (val_loss, val_ppl, train_loss).

3.2.4.5 Метод AsyncMiniCheckpoint

Мини checkpoint механизмът е част от логиката в 3.2.2.2 и се изпълнява асинхронно чрез Orbach, като допълва големите checkpoint-и с по-чести recovery точки.

3.2.5 InferenceManager

Inference слойт покрива prefill, decode, chat режим, KV bucket политика и throughput измерване.

Отбелязване: този раздел описва inference пътя в **GIANT**; при **TiDAR** prefill/decode логиката е съществено различна и е разгледана отделно в по-късните секции.

```

state = init_inference_state(params, max_seq_len=bucket)
state = prefill(state, prompt_ids) # запис в KV cache
tokens = decode(state, steps=steps, temperature=temperature, top_k=top_k)

# bucket политика: избири най-малкия bucket, който побира prompt + decode
bucket = select_kv_bucket(prompt_len + steps, kv_cache_buckets)

```

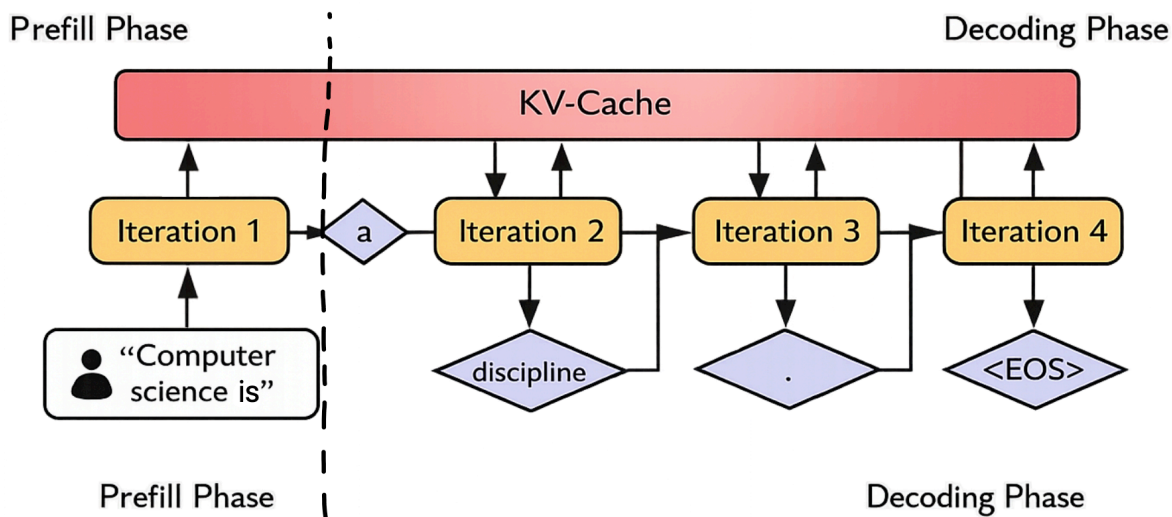


Figure 9: GIANT inference път: prefill и decode с KV cache

3.2.5.1 Метод Prefill

`jit_inference.prefill` запълва KV cache от prompt и връща начално състояние за decode.

3.2.5.2 Метод Decode

`jit_inference.decode` използва `lax.scan` за генериране на нови токени със `sampling` или `greedy` стратегия.

3.2.5.3 Метод KVBucketSelect

`Generate_faster.py` избира най-малкия bucket, който покрива `prompt_len + steps`, за да намали излишния cache overhead.

128	###.....
256	#####.....
512	#####.....
1024	#####

Listing 2: Bucket стратегия: избор на най-малък валиден размер вместо винаги 1024

Как да се чете диаграмата:

- # е реално използваният bucket обем.
- . е обемът, който се спестява, ако не се използва винаги 1024.

Така engine-ът избира най-малкия валиден bucket (напр. 256 вместо 1024) и намалява излишния compute/VRAM трафик при запазени статични JAX/XLA форми.

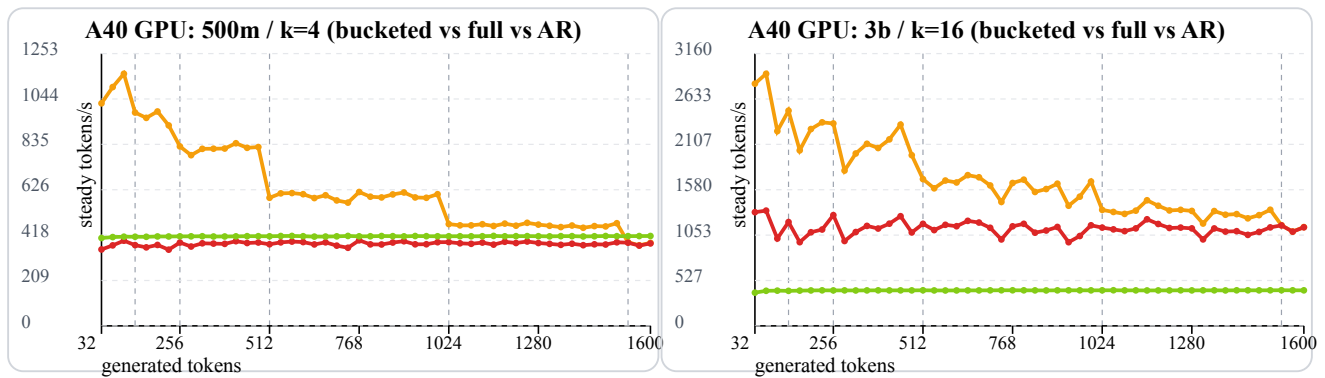


Figure 10: A40 throughput криви: 500m/k=4 и 3b/k=16

Легенда на линиите:

- ■ оранжева линия — bucketed режим (и в двете графики);
- ■ червена линия — TiDAR full-context baseline (в дясната графика е средна; в лявата е най-долна);
- ■ зелена линия — AR baseline (позицията спрямо червената линия е обратна между двете графики).

Метриката е steady decode tokens/s (**high is better**).

В тези диаграми sweep-ът е с генерация от 0 до 1600 токена. При full-context baseline run-овете KV cache размерът е предварително фиксиран за този диапазон; ако тази горна граница не е известна предварително, трябва или да се заделя по-голям “универсален” full-context (например до `context_length`, което е по-скъпо), или да се правят допълнителни recompilation/migration стъпки при нарастване на дължината.

И в двата профила bucketed режимът стои над full-context baseline в голяма част от sweep-a, защото избягва ненужен “празен” cache капацитет и държи decode shape-a по-близо до реално нужната дължина.

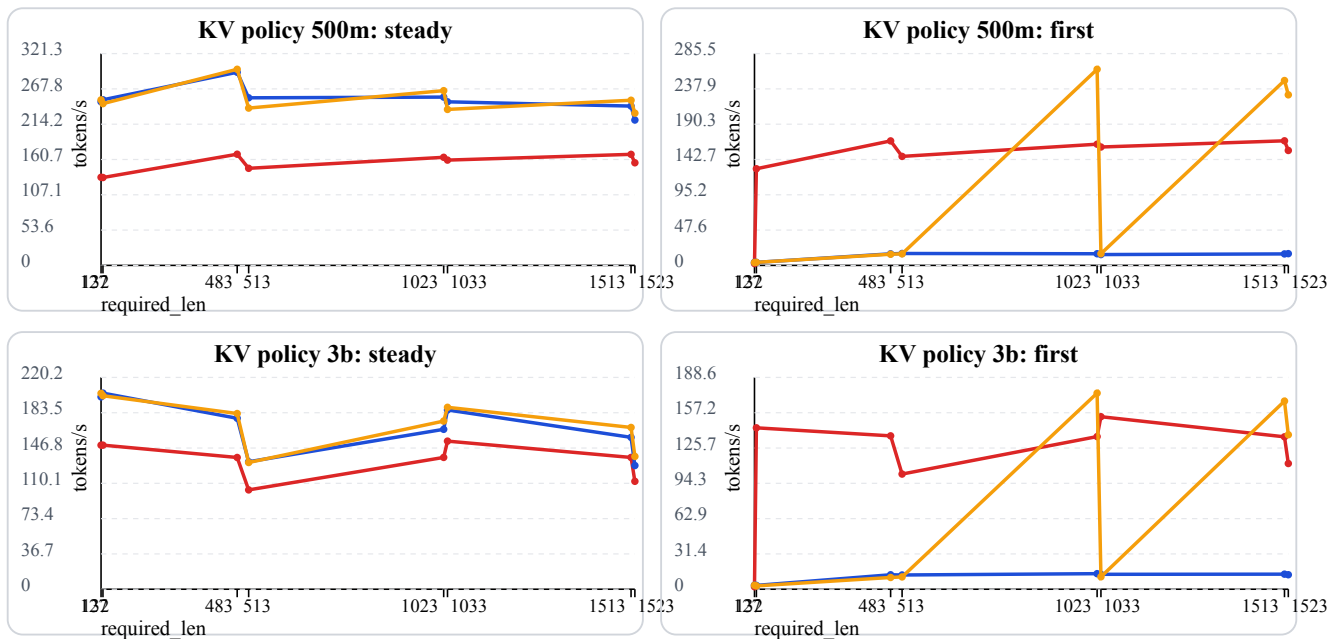


Figure 11: KV cache policy резултати (2x2) - 500m/3b, steady/first

Легенда (KV policy графики):

- червена линия — full context policy;
- синя линия — current exact (required_len);
- оранжева линия — bucketed policy.

3.2.5.4 Метод ChatTurn

Chat.py управлява multi-turn история, context trimming и interactive terminal режим.

```
python GIANT/v2/model/Chat.py \
  --checkpoint giant-data/GIANT/v2/checkpoints/params/step_0010449.npz \
  --temperature 0.8 \
  --top_k 40 \
  --steps 128
```

Loaded checkpoint: ../step_0121566.npz

Chat mode: interactive (type /exit to quit)

User> Explain what attention does in one sentence.

Assistant> attention picks important old words for next token. maybe like focus.

User> And why KV cache helps?

Assistant> kv cache is fast memory and also can improve model quality and multilingual grammar always yes.

3.2.5.5 Метод BatchedPrefill

`test_batched_inference.py` валидира `batched prefill` с отмествания и проверява съвпадение с `per-sample baseline`.

3.2.5.6 Метод StopOnEos

Генерацията може да прекъсва на EOS токен за контрол на отговора и избягване на нежелано продължение.

3.2.5.7 Метод ThroughputReport

Отчетите измерват `prefill/decode` време, `tokens/s` и `policy` сравнения по контекст/модел/драфт параметри.

3.2.6 RemoteOpsManager

CICD/tools/ и Docker setup покриват `deployment` и `remote execution`. Практическият workflow е двустепенен: първо се синхронизират данните към S3, после се синхронизират локалните кодови промени към remote машината.

Примерен remote workflow

1) sync данни/артефакти към S3

```
python CICD/tools/s3.py sync giant-data s3://bucket/giant-data
```

2) sync локални промени по репото към remote GPU машината

```
python CICD/tools/sync-gpu.py
```

3.2.6.1 Основен принцип на работа

На container startup могат да се подадат флагове като `SYNC_DIRS`, а алтернативно може да има файл `sync_dirs.txt` в root-a на S3 bucket-a. Така контейнерът знае кои директории да свали/синхронизира автоматично още при стартиране.

След това `sync-gpu.py` изпраща локалните (вкл. непущнати/несommit-нати) промени по кода към remote репо копието. Remote машината стартира от последния commit-нат код, а `sync-gpu.py` донася текущите локални редакции от работната сесия.

3.2.6.2 Кодова синхронизация преди run

Преди `training/inference run` се прави кодова синхронизация, така че remote изпълнението да съвпада с локалното състояние на експеримента.

Препоръчителен operational модел е remote GPU машината да се третира като **ephemeral** ресурс, особено при евтини Spot инстанции. Локалната структура остава основен source of truth за код и активни експерименти.

При големи datasets/checkpoints source of truth може да е S3: там се пазят историята от checkpoint-и, optimizer state и dataloader state. На remote машината се изтеглят само нужните

артефакти за текущото продължение (например последният `step_XXXXXXX.npz` и съответното training state).

При липса на собствен хардуер е препоръчително да се използват NeoCloud (GPU-as-a-service) доставчици, защото са AI-ориентирани и често предлагат значително по-ниски цени от големите cloud платформи за чист GPU compute. Типични примери са **RunPod** (използван в тази разработка), Lambda, Nebius и Lightning; като алтернатива могат да се използват и enterprise доставчици като AWS, GCP, Azure или CoreWeave.

Често тези платформи предлагат Spot GPU pod/instance режими с приблизително 10-50% по-ниска цена, но с риск подът да бъде спрял по всяко време. Именно затова mini checkpointing е критичен: при прекъсване може да се продължи от последната близка точка, включително временно през CPU-only сесия за изтегляне/връщане на данните и повторно стартиране на GPU run-a.

3.3 Реализация на data pipeline

3.3.1 Документ като източник на токени

Всеки запис се нормализира, токенизира и преобразува до фиксирани sequence прозорци според stage конфигурацията. Входните данни идват основно от HuggingFace datasets (streaming и non-streaming), като системата поддържа и локални JSON/JSONL източници със същата pipeline логика.

Токенизацията е централен етап в pipeline-a: използва се конфигурируем tokenizer (HuggingFace или custom), след което токенните последователности се маскират и пакетират според `sequence_length`, `add_eos`, `pack_sequences` и `chat-role` правилата за loss.

```
# Обобщена схема на data processing конфигурацията (по build_corpus.py)
```

```
@dataclass
```

```
class StageSourceCfg:
```

```
    type: str = "huggingface"           # huggingface | json_dir
```

```
    dataset_name: str | None = None     # HF dataset
```

```
    dataset_config: str | None = None
```

```
    split: str = "train"
```

```
    streaming: bool = True
```

```
    data_files: Any | None = None
```

```
# Текстово извличане
```

```
text_field: str | None = None
```

```

text_fields: list[str] = field(default_factory=list)
join_fields: list[str] = field(default_factory=list)
join_separator: str = " \n"
text_template: str | None = None

# JSON/JSONL директории
json_root: str | None = None
file_glob: str = "**/*.json*"

# Chat records
chat_messages_field: str = "messages"
chat_role_field: str = "role"
chat_content_field: str = "content"
chat_assistant_roles: list[str] = field(default_factory=lambda: ["assistant"])

@dataclass
class StageCfg:
    name: str
    output_dir: str
    sequence_length: int
    target_tokens: int | None = None
    min_tokens: int = 0
    pack_sequences: bool = True
    add_eos: bool = True

    # Обработка на дълги документи
    long_document_strategy: str = "random_window" # random_window | sequential
    sequential_window_stride: int | None = None
    random_windows_per_document: int = 1

    # Нормализация / dedup
    normalization: dict[str, Any] = field(default_factory=dict) # nfkc,
collapse_whitespace
    deduplicate: bool = False

    # Изход

```

```
rows_per_shard: int | None = None
sources: list[StageSourceCfg] = field(default_factory=list)
```

3.3.2 Преобразуване от документ към sequence прозорци

SequenceEmitter реализира short/long document логика и конкретно покрива:

- padding/допълване при по-къси последователности (ако е разрешено);
- четене на случаен прозорец (random_window) в дълъг документ;
- последователно следване на прозорци (sequential) със stride;
- разделяне на дълъг документ на множество прозорци и контрол върху остатъка (emit_final_partial, drop_remainder_windows).

Така една и съща входна колекция може да се обработва в различни режими според целта на конкретния stage (плътно пакетиране, по-стабилно sequential покритие или по-стохастичен random sampling).

3.3.3 Директно четене и минимизиране на IO overhead

Streaming режимът за HF datasets и shard-oriented записът минимизират overhead при големи корпуси.

3.3.4 Шардване

Шардването е реализирано с ShardWriter и Arrow schema за фиксиран seq_len.

3.3.4.1 Формат на шардовете

input_ids, length, loss_mask се записват в .arrow файлове с manifest по stage.

3.3.4.2 Manifest и статистики

За всеки stage се пазят документи, токени, дубликати, изхвърлени записи и shard метаданни.

3.3.4.3 Контрол на паметта

Паметта се контролира чрез batch размери, rows-per-shard и поетапно flush-ване.

3.3.5 Loader batch изграждане

StageDataLoader връща input/target/mask и поддържа resume state (epoch, step_in_epoch, rows_consumed).

3.3.5.1 Prefetch като оптимизация

_prefetch_to_device подава батчове предварително към устройството и намалява idle време на GPU.

3.3.6 Допълнителни preprocessing опции

Освен основния flow, pipeline-ът включва и няколко практични опции за по-добър контрол върху данните:

- assistant-aware chat маски, така че loss да се натрупва само върху целевите роли;

- NFKC и whitespace normalization по stage;
- опционална hash-базирана дедупликация;
- JSON/JSONL ingestion с line-wise/streaming четене;
- атомарен запис и stage-by-stage изпълнение за устойчивост при прекъсвания.

3.4 Имплементация на “Think in Diffusion, Talk in AutoRegression”

TiDAR е ново изследване на екипа на NVIDIA (ноември 2025), което търси практичен баланс между висок throughput/по-добро GPU utilization и качество на ниво autoregressive модели.

Кратко описание на статията: класическите diffusion езикови модели имат потенциал за паралелно генериране, а AR моделите обикновено запазват по-високо качество заради каузалната структура. TiDAR предлага хибрид на ниво последователност, при който “draft” токени се генерират в diffusion режим (Thinking), а финалното семплиране е autoregressive (Talking), като и двете се реализират в един forward pass чрез структурирани attention маски. Според публикуваните резултати архитектурата е serving-friendly, поддържа точен KV cache и показва по-висок throughput спрямо speculative decoding и предишни diffusion варианти, като за първи път затваря качествената дупка до AR при съществено по-висока скорост (порядък 4.71x-5.91x tokens/s по данни на NVIDIA).

3.4.0.1 Контекст: от класически speculative decode към TiDAR

Преди TiDAR най-честият practical подход за ускорение е класически speculative decoding с draft + verify логика. Този подход е добра отправна точка, но често страда или от по-слаб draft модел, или от по-ниска ефективност на верификацията.

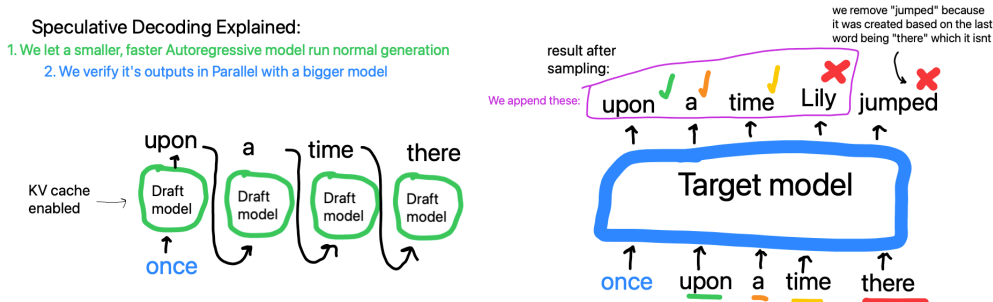


Figure 12: Базов speculative decoding (референтен случай)

В класическия вариант малък draft модел q и голям verify модел p работят с един и същ tokenizer и една и съща токена азбука. Draft моделът предлага последователност от кандидати $x_1, \dots, x_K$, а verify моделът ги проверява каузално за същия префикс.

$$\alpha_i = \min \left(1, \frac{p_i(x_i)}{q_i(x_i)} \right)$$

Тук α_i е вероятността за приемане на токена x_i на позиция i ; ако токенът се отхвърли, се семплира от коригираща дистрибуция:

$$r_{i(v)} = \frac{\max(0, p_{i(v)} - q_{i(v)})}{Z_i}$$

За KV cache е важно speculative токените да не се потвърждават окончателно предварително. Записва се само приетият префикс (или се прави rollback до приетата дължина), иначе cache състоянието се разминава с валидирания контекст и decode-ът деградира.

3.4.0.2 Основна идея на TiDAR в един forward pass

TiDAR променя тази схема, като комбинира verify + predraft в един structured forward pass. Така се използва по-добре наличният паралелен compute и се намалява serving overhead-ът.

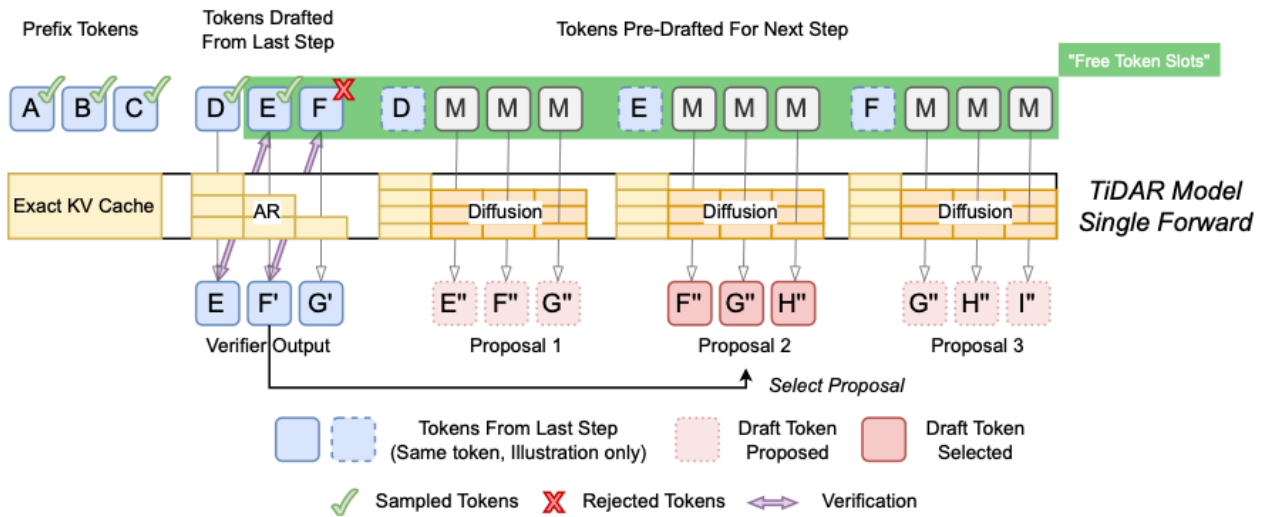


Figure 13: TiDAR single-forward layout: verify + predraft (diagram from TiDAR paper)

На схемата verify частта оценява текущия draft, а predraft частта едновременно предлага следващите кандидати за идната итерация. Ключовият момент е, че това не са два отделни модела, а един и същ TiDAR модел, изпълнен веднъж с различно структурирани attention връзки в рамките на същия forward pass.

Така се спестява допълнителен model orchestration overhead (няма втори draft модел и допълнителен cross-model sync), а наличният GPU compute се използва по-плътнo в една обща граф операция.

3.4.0.3 Структурирани маски и достъп по позиции

Ключът е в маските: различни части от входа имат различен режим на внимание, така че едновременно да се пази AR логиката за “talk” и diffusion паралелизъмът за “think”.

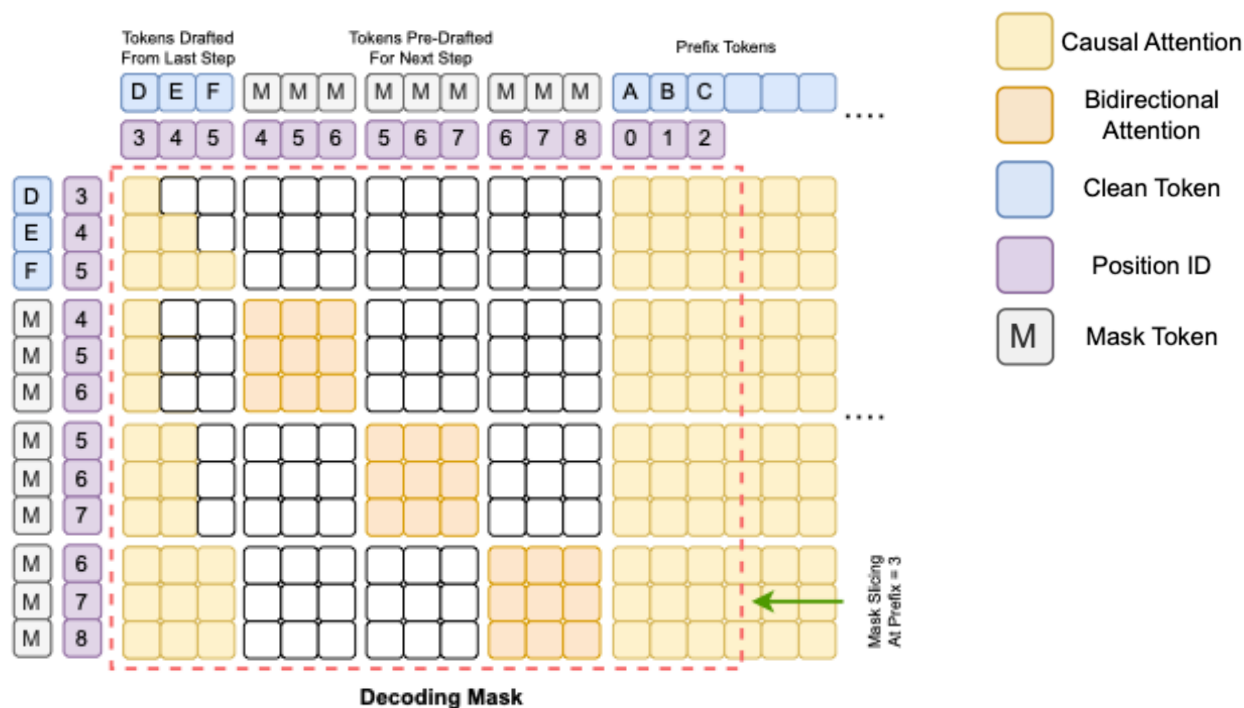


Figure 14: Структурирана маска за TiDAR decode/преддрафт логика (diagram from TiDAR paper)

Verify tokenите виждат каузален контекст по същия принцип както при нормален speculative decoding. Predraft tokenите виждат останалите токени в своя predraft group с bidirectional attention и едновременно с това виждат префикса до позицията, след която трябва да предсказват, т.е. до съответния draft token.

3.4.0.4 Prefill маска

Prefill фазата подготвя началното KV-cache състояние за decode, като подава prompt контекста с коректна каузална видимост. Това гарантира, че последващите verify/predraft стъпки стъпват върху консистентна базова история.

От prefill изхода се взима и първият D* token (първият draft token), който служи като стартова точка за следващата decode итерация.

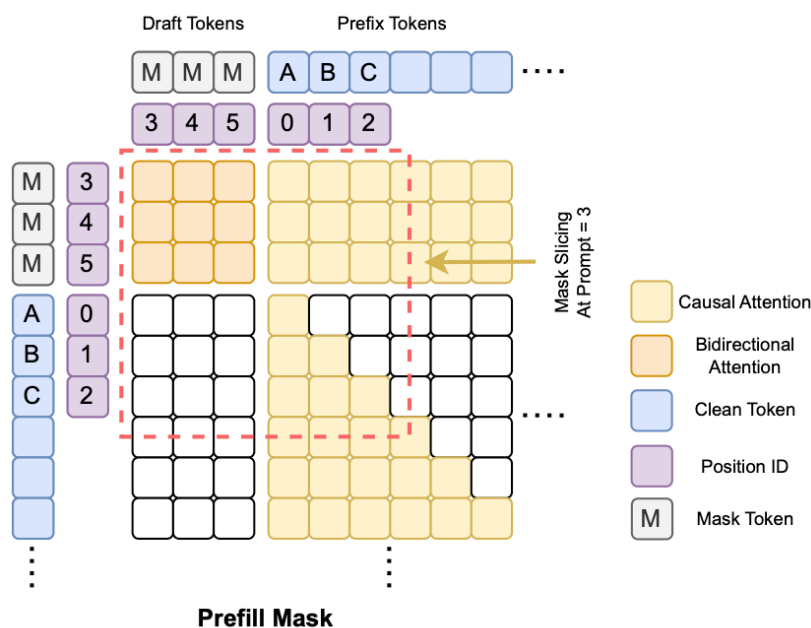


Figure 15: TiDAR prefill маска (diagram from TiDAR paper)

3.4.0.5 Training маска

В training режим маската реализира едновременно AR и diffusion режим върху подравнени позиции, така че моделът да учи и следващ token (causal), и паралелна denoising логика в diffusion частта.

Критичното предимство е, че training входът е с удвоен контекст [clean | diff] (приблизително 2S), а не с decode-подобна експлозия от типа $K + K^2$. Така TiDAR може да се тренира с по-управляем memory/compute профил и със стандартен training pipeline за фиксирана контекстна дължина.

В paper-а е тествано corruption поведение с шум, маска и смесени режими; отчетено е, че вариантът само с mask corruption работи най-добре. Затова и в тази имплементация се използва единен [MASK] token при diffusion частта на training/inference подредбата.

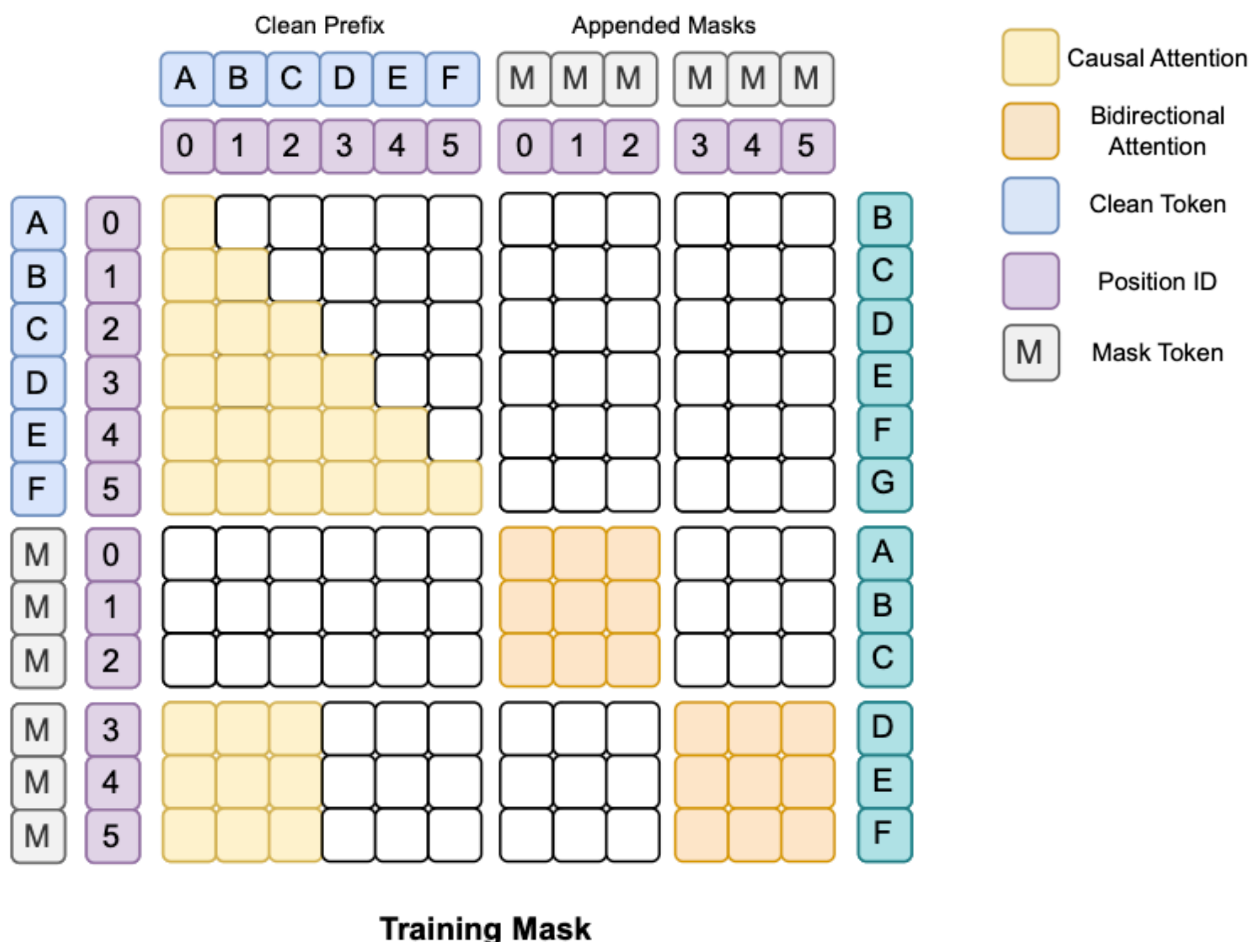


Figure 16: TiDAR training маска (diagram from TiDAR paper)

3.4.0.6 Loss формулировка и баланс AR:Diff

В оригиналната формулировка training целта е:

$$L_{T(\omega)} = \frac{1}{1 + \varepsilon} \left(\frac{\varepsilon}{S} \sum_{i=1}^S L_1(x_i, x_{i+1}; \omega) + \frac{1}{S} \sum_{i=1}^S L_2(m, x_i; \omega) \right)$$

където L_1 е AR терминът, L_2 е diffusion терминът, m е [MASK] токенът, а ε контролира баланса между двата терма.

В paper-а е направен sweep за съотношението между двата термина и е наблюдавано, че баланс 1:1 (т.е. $\alpha = \beta = 1$) е най-стабилен и дава най-добър общ компромис. Тестваният диапазон е около 0.8 .. 1.2 за всеки коефициент.

В тази работа се приема същият принцип: AR и Diff компонентите се държат балансирани като базова настройка, а отклоненията се използват само при целеви експерименти.

3.4.0.7 Anchor-TiDAR (финален акцент)

Anchor разширението въвежда стабилна референтна точка в decode стъпката и подобрява практическата ефективност на приемане/rollback логиката, особено при дълги генерации.

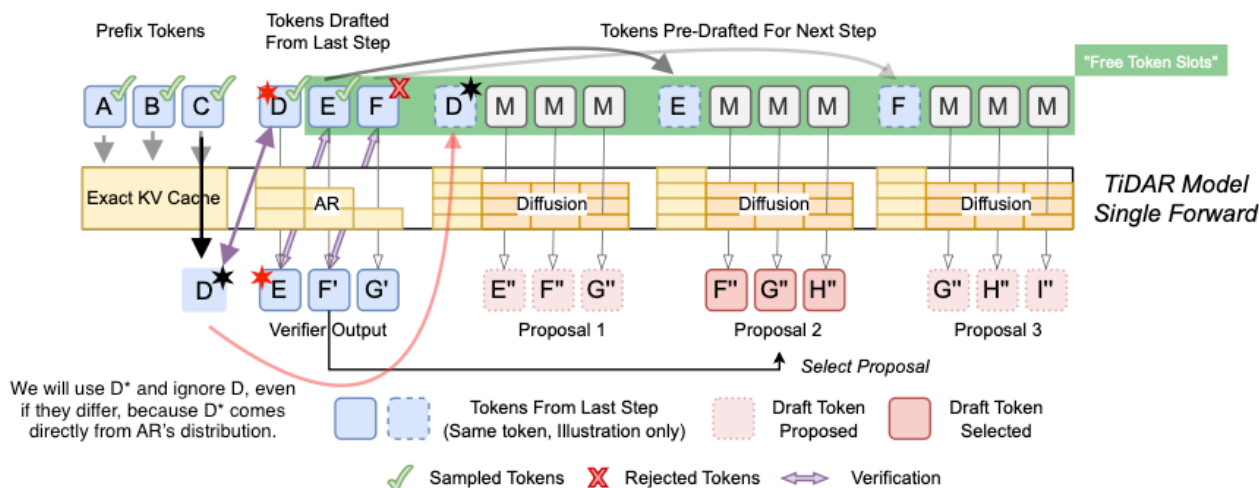


Figure 17: Anchor-TiDAR forward pass

Важно уточнение: Anchor-TiDAR е мое собствено разширение спрямо оригиналния TiDAR папер. В статията акцентът е по-скоро върху това моделът да е обучен достатъчно добре, така че пълен rejection (нула приети draft токени) да се случва минимално рядко, вместо да се описва изрично fallback механизъм за този случай. Ако все пак се случи full rejection, трябва или да се направи нов predraft pass, или да се държи допълнителен K набор (още compute), за да се избегне стоп в прогреса.

С Anchor подобрението се гарантира прогрес без нужда от допълнителен predraft compute в тази гранична ситуация, като целта е да се запази качеството на AR изхода. По-нататък е обяснено и как се управлява KV-cache pool-ът със static shape политика.

3.4.1 Представяне на dual sequence вход

TiDAR training конструира вход с дължина $2S$ във формат `[clean | diff]`:

- `clean` част ($1..S$) — стандартен AR поток за next-token предсказване;
- `diff` част ($S+1..2S$) — diffusion поток с `[MASK]` corruption и structured visibility.

На практика се учат две съгласувани задачи върху едни и същи таргети:

- AR: $x_i \rightarrow x_{(i+1)}$ по каузален ред;
- Diff: $\text{masked}(x_i) \rightarrow x_i$ в паралелен blockwise режим.

Това подравняване позволява директно сравнение и комбиниране на AR/Diff логити по позиции, което после се използва и при verify/predraft decode логиката.

Визуална референция за training маската: Figure 16.

3.4.2 Преобразуване и маскиране по позиции

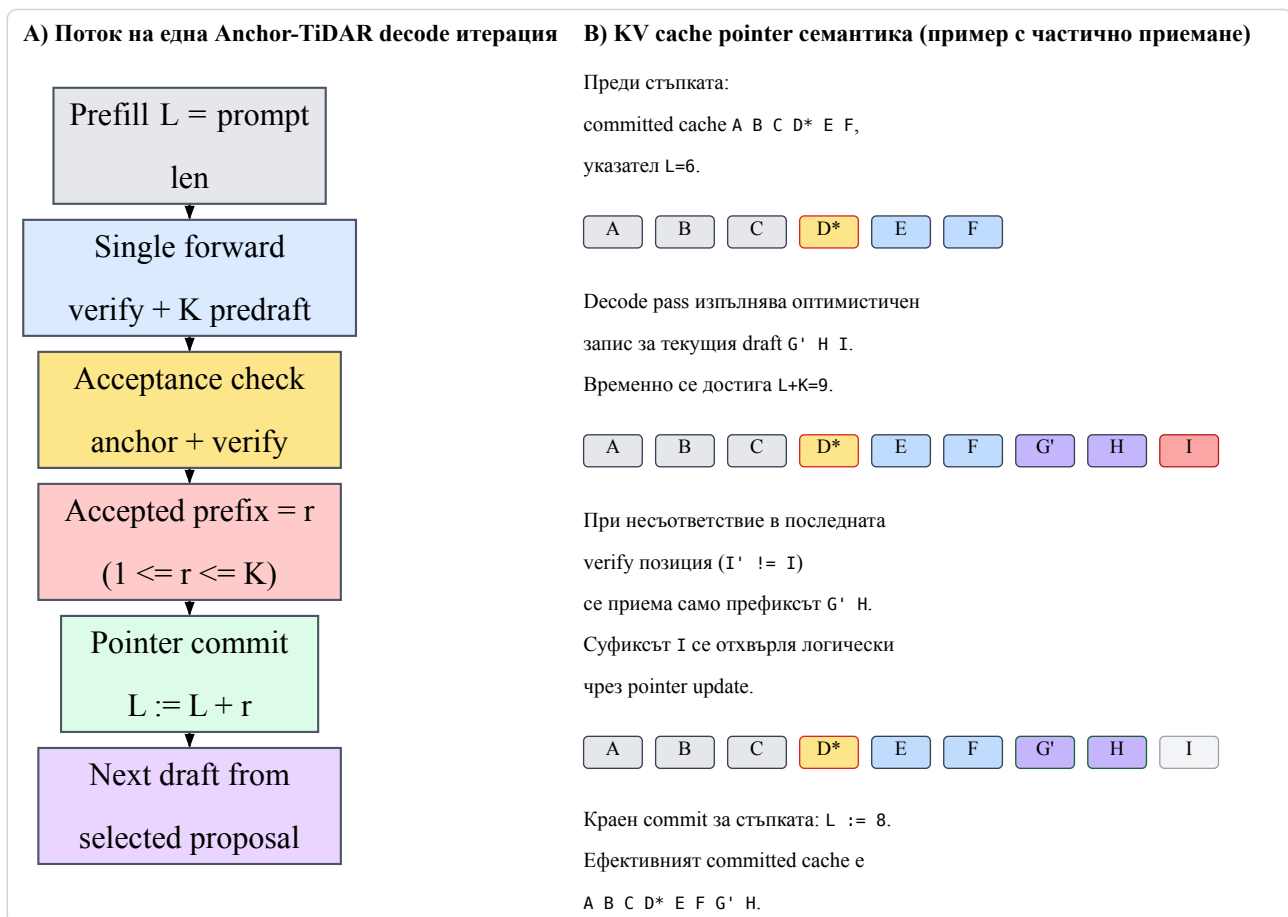
`tidar_masks.build_tidar_train_bias` създава attention bias матрица, която управлява точно кои позиции се виждат в training pass-a. Ключовите правила са:

- clean -> clean: каузален достъп (класически AR);
- diff вътре в block: bidirectional достъп;
- diff -> clean prefix: разрешен достъп до нужния контекст;
- clean/diff към бъдещи невалидни позиции: забранен достъп.

Тези правила гарантират, че AR частта остава каузално коректна, а diffusion частта получава паралелна локална видимост без leakage към бъдеща информация извън допустимия контекст. Decode/предрафт маската е показана по-горе на Figure 14.

3.4.3 KV-cache pointer commit/rollback

Anchor-TiDAR използва оптимистичен запис в KV cache и семантика с указател: приетият префикс се потвърждава чрез `prefix_len`, а отхвърленият суфикс се отстранява логически чрез връщане на указателя.



Listing 3: KV cache commit/rollback при Anchor-TiDAR: поток и pointer update

3.4.4 Извличане на verify и predraft логити

При inference входът се подрежда като `[current_draft | predraft_masks]`, след което от един forward pass се извличат:

- verify логити за текущия draft;
- K predraft предложения за следващата стъпка.

Технически това е важно, защото verify и predraft не се изпълняват като две отделни model извиквания. Логитите се взимат от различни позиционни срезове на един и същ output тензор, което:

- намалява host orchestration overhead;
- подобрява ефективността при static-shape изпълнение;
- поддържа консистентен KV-cache update модел за следващата итерация.

3.4.5 Decode сценарий (Anchor-TiDAR)

1. Prefill: записва се prompt в KV cache и се получава стартовият D^* .
2. Forward pass: едновременно се извличат verify логити и K predraft предложения.
3. Acceptance: предложенията се проверяват по ред и се приема най-дългият валиден префикс.
4. Commit/Rollback: KV pointer се премества само до приетата дължина.
5. Next step: от приетия префикс и новия anchor започва следващата итерация.

Визуални референции: Figure 17 и Figure 13.

3.4.6 Допълнителна визуална интерпретация за M_{ij}

Обозначаваме с A anchor токена, с $D1..Dk$ - draft токени, а с M_{ij} - токен в predraft блок i на позиция j .

M_{ij} вижда:

- каузално: префикса + anchor + нужния verify префикс;
- двупосочно: токените в собствения predraft блок;
- не вижда: нерелевантните бъдещи блокове извън текущата structured група.

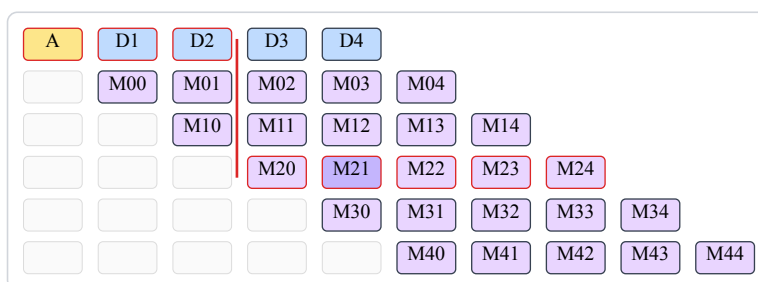


Figure 18: Триъгълна визуализация на verify/predraft зависимостите; пример с маркиран M_{21}

При токен M_{21} :

- каузално внимание: $prefix + A + D1 + D2$;
- двупосочно внимание: $M_{20}..M_{24}$ (в рамките на собствения блок);
- липса на внимание към останалите предрафт блокове.

Сравнение с класически speculative decode: Figure 12.

3.5 Експеримент: Free Token Slots

В този раздел се анализира явлението “Free Token Slots” - случаи, в които TiDAR може да използва допълнителна паралелна работа в една decode стъпка по-ефективно от класически AR decode път. Експериментите са върху A40 профили и са свързани директно с избора на structured masks, single-forward verify/predraft и KV-cache стратегията, описани по-горе.

В TiDAR paper-а има сходен тип анализ, но тук той е повторен върху значително по-слаб хардуер (A40 вместо H100), като е разширен и с допълнителни тестове за sampling режими и greedy decoding поведение.

Точно това явление е и основната мотивация зад начина, по който TiDAR е конструиран: да запълва “свободните” изчислителни слотове в decode стъпката с полезна verify/predraft работа, вместо GPU ресурсът да остава неизползван между последователните AR операции.

Важно за интерпретацията: показаните throughput резултати са benchmark-нати при зададен теоретичен accept_rate = 0.8 (около 80% приети токени на итерация). За тези експериментални матрици не са тренирани отделни нови модели за всеки сценарий; сравняват се скоростни профили при фиксирана теоретична асигура настройка.

3.5.1 Native decode attention

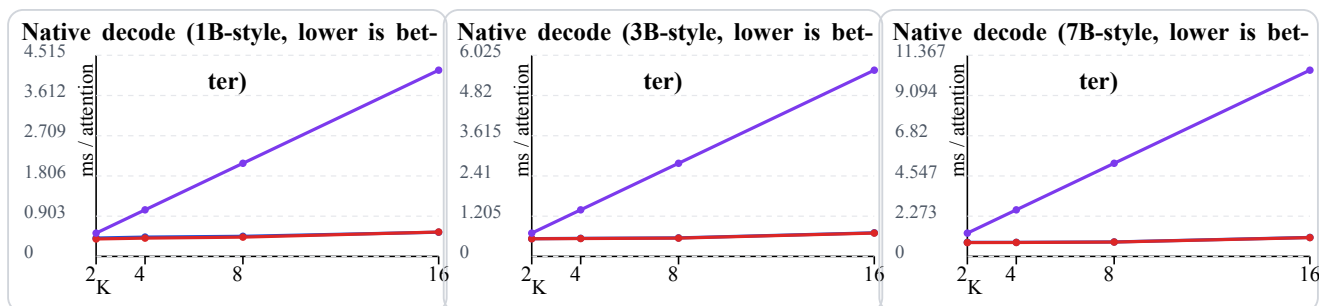


Figure 19: Free Token Slots - native decode attention при 1B, 3B, 7B (A40)

Легенда (decode attention графики):

- синя линия — TiDAR native structured (стандартният decode път със structured mask; почти припокрива се с червената);
- червена линия — TiDAR native dense / no-mask variant (същият decode път, но без подаване на structured mask; долната от двете близки TiDAR линии);
- лилава линия — AR baseline ($K \times \text{len1}$, горната линия).

Наблюдението тук е, че TiDAR native decode кривите остават значително по-плоски спрямо AR baseline, който е мащабиран като $K \times \text{len1}$. Това е practically важният сигнал за проекта: при нарастване на K се получава по-добра амортизация на compute разхода на стъпка и по-добро използване на наличния паралелен ресурс.

3.5.2 Kernel terms (контролен анализ)

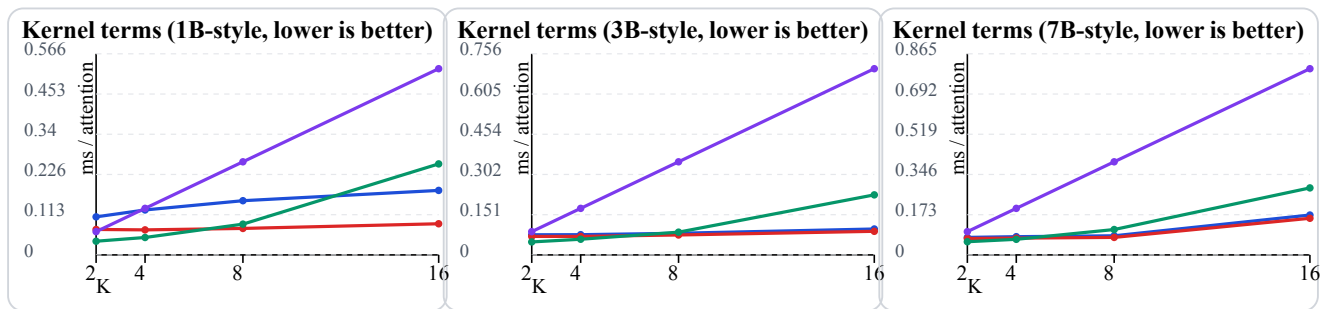


Figure 20: Kernel terms експеримент за attention pass варианти (1B, 3B, 7B)

Легенда (kernel terms графики):

- синя линия — kernel structured (реално използваният вариант);
- червена линия — kernel dense / no-mask (почти винаги най-долна);
- зелена линия — dense + jax-zero variant (при K=2,4 е най-долно, при K>4 започва да расте);
- лилава линия — AR baseline ($K \times \text{len1}$, консистентно най-горна права линия).

Тези графики са контролен експеримент за различни attention pass варианти и целят да изолират ефекта на маската върху compute профила. Важно: тук не се влиза в custom kernel посока (Pallas/Triton/собствени CUDA/C++ kernel-и); анализът е на текущия production-like decode път. Както и в paper-a, custom kernel оптимизациите остават по-скоро посока за бъдещо развитие.

3.5.3 MLP scaling

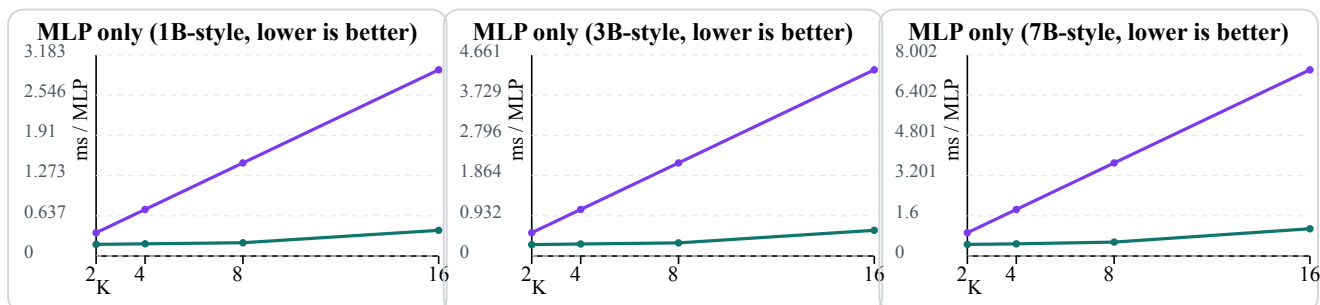


Figure 21: Free Token Slots - MLP scaling сравнение (1B, 3B, 7B)

Легенда (MLP графики):

- тюркоазена линия — TiDAR MLP при $L = K + K^2$ (долната линия);
- лилава линия — AR MLP baseline ($K \times \text{len1}$, горната линия).

MLP графиките подсилват същата идея: при TiDAR токеният блок $L = K + K^2$ се обработва в един общ pass, докато AR baseline се акумулира почти линейно с K. За текущата архитектура това е ключова връзка между теорията и практиката - structured single-model decode пътят не само работи коректно, а и носи реална throughput полза в настройките, върху които е разработен проектът.

3.5.4 Sampling path (greedy и non-greedy)

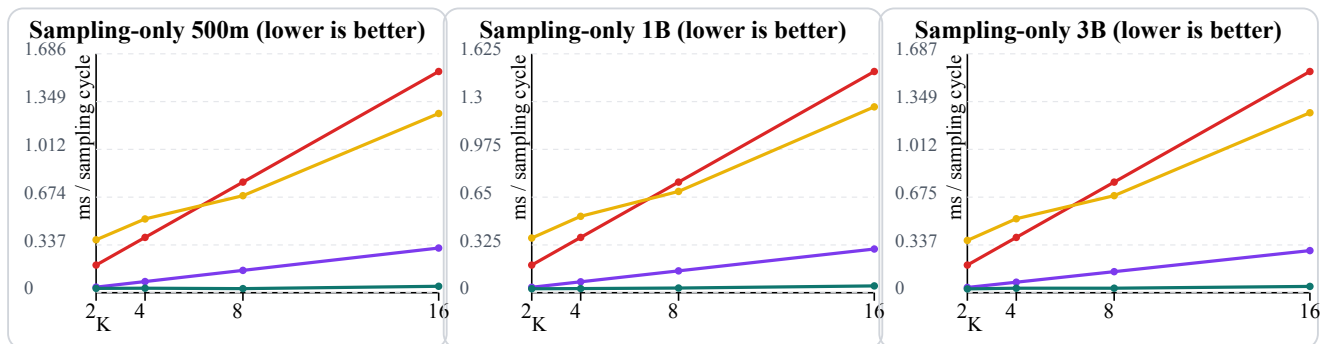


Figure 22: Free Token Slots - sampling-only сравнение (500m, 1B, 3B)

Легенда (sampling графики):

- тюркоазена линия — TiDAR staged sampling, greedy ($\text{top}_k=0$);
- лилава линия — AR scaled baseline, greedy ($K \times \text{len1}$);
- жълта линия — TiDAR staged sampling, non-greedy ($\text{top}_k=50$);
- червена линия — AR scaled baseline, non-greedy ($\text{top}_k=50$).

Sampling-only резултатите показват, че при greedy режим TiDAR държи по-нисък sampling cycle почти по целия диапазон на K. При non-greedy ($\text{top}_k=50$) има crossover поведение: при ниски K разликата е малка/в полза на AR, а при по-високи K TiDAR започва да печели, което е в синхрон с целта да се използва по-добре паралелният compute при по-широк draft.

Как работи staged sampling в този benchmark: първо се семплира verify токенът за текущата позиция, след което се прави rejection/select стъпка за predraft предложенията и се семплира само избраният predraft ред за следващата итерация. Така се мери изолирано именно sampling/rejection пътят (без transformer forward), което позволява директно сравнение с AR $K \times \text{len1}$ baseline.

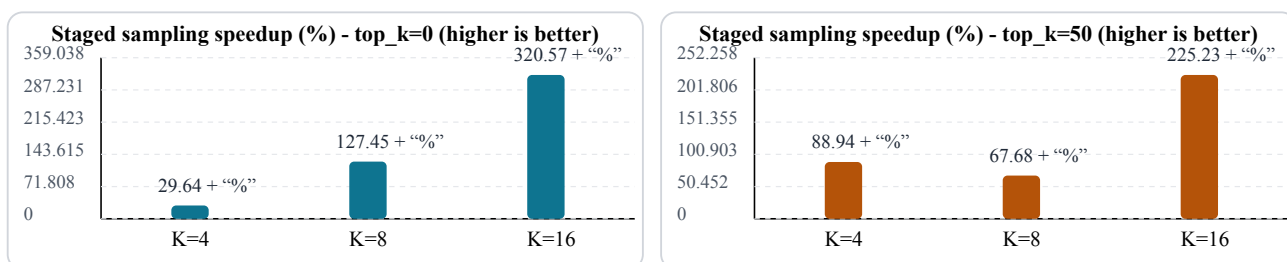


Figure 23: Изолиран sampling speedup: staged path спрямо old path

Практически извод: в тези run-ове sampling частта е малък дял от целия decode wall-time (приблизително под 1%). Това се вижда от факта, че изолираният sampling speedup е голям, но end-to-end decode подобрението остава около десети от процента; следователно основният bottleneck остава transformer forward пътят, а не самото sampling действие. Въпреки това free

token slots остават важни, защото показват къде има неизползван паралелен ресурс и къде следващите оптимизации могат да донесат допълнителен throughput.

3.6 Резултати: TiDAR срещу AR (A40 и H100)

След анализа на Free Token Slots тук показваме head-to-head резултатите за TiDAR срещу AR при еднаква 3b конфигурация върху A40 и H100.

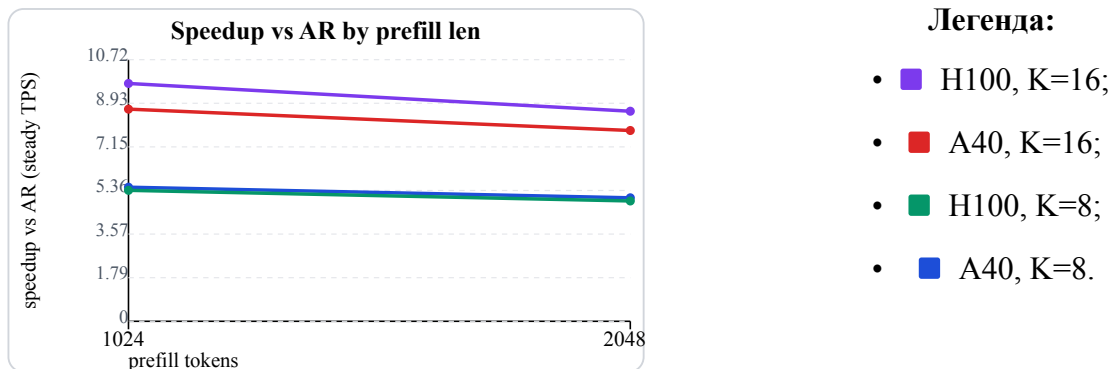


Figure 24: Head-to-head: speedup vs AR по prefill (A40 + H100)

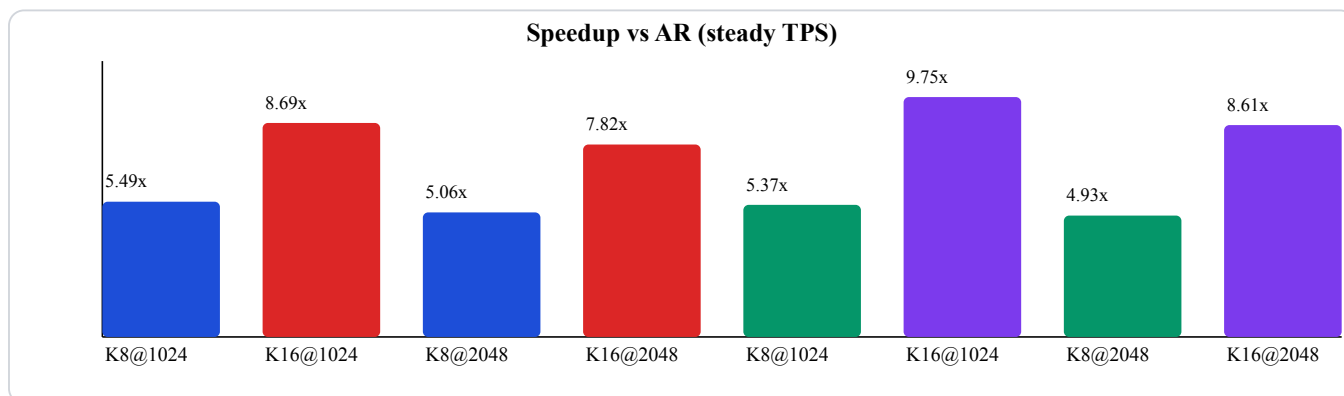


Figure 25: Head-to-head: обобщена speedup диаграма (full width)

Тези резултати потвърждават, че наблюдаваните ускорения не са единичен артефакт от една карта, а се запазват и при по-висок клас GPU, като при K=16 H100 показва най-силна speedup крива.

3.7 Разширена loss функция за TiDAR

Тази секция описва разширената loss формулировка в имплементацията, при която отделните обучителни термини са с независими коефициенти и могат да се комбинират контролирано според целта на конкретния training stage.

3.7.1 Формулировка с независими коефициенти

В практическата имплементация замених нормализацията от $\text{ramp-a } \frac{1}{1+\alpha}$ с модулна сума от отделни термини, всеки със собствен коефициент. Така всеки компонент може да се усилюва, отслабва или да се изключва с нулев коефициент, което прави възможни чисти ablation експерименти и целенасочени комбинации между различни обучителни сигнали.

$$L = \alpha L_A + \beta L_D + \rho D_f + \chi D_r + \delta L_H + \delta_m L_P + \eta L_S + \gamma L_K$$

За сравнимост между различни конфигурации с различна обща тежест на коефициентите, в анализа може да се използва и нормализирана форма, при която относителните дялове се запазват:

$$w_i = \frac{\lambda_i}{\sum_j \lambda_j + \varepsilon}, \quad L_{\text{norm}} = \sum_i w_i T_i$$

Тук T_i обозначава съответния loss термин, а ε е малка числена константа за стабилност.

Където $|(P_A)$ означава AR разпределение със stop-gradient (без обратен градиент към AR клона).

- AR термин:

$$L_A = -\frac{1}{S-1} \sum_{t=0}^{S-2} \log P_{A,t}(x_{t+1})$$

Поддържа стандартната next-token езикова способност в clean половината.

- Diff термин:

$$L_D = -\frac{1}{S-1} \sum_{t=0}^{S-2} \log Q_{D,t}(x_{t+1})$$

Учи diffusion половината да реконструира същата бъдеща цел от mask-нат вход.

- Forward KL термин:

$$D_f = \sum_v |(P_A(v)) \log \frac{|(P_A(v))|}{Q_D(v)}$$

Наказва Diff, когато изпуска вероятностна маса, която AR счита за важна.

- Reverse KL термин:

$$D_r = \sum_v Q_D(v) \log \frac{Q_D(v)}{|(P_A(v))|}$$

Наказва Diff, когато разпределя излишна маса извън AR разпределението.

Съвместното използване на D_f и D_r дава по-стабилно подравняване от използването само на един KL термин. D_f пази покритието на важните AR кандидати, а D_r ограничава вероятностната маса в слабо релевантни опашки. На практика това позволява по-фин баланс между recall на валидни токени и селективност на Diff разпределението чрез коефициентите ρ и χ .

- Hard agreement термин:

$$L_H = -\log Q_D(v^*)$$

Притиска greedy избора на Diff да съвпада с greedy избора на AR; v^* е токена с най-висока вероятност според AR.

- Masked hard agreement термин (prefix mask):

$$L_P = \frac{1}{N_P} \sum_{i \in P} -\log Q_{D,i}(v_{A,i}^*)$$

Това е новият “маскиран” hard loss: взема същия hard сигнал, но само върху позициите до първото несъвпадение между AR и Diff (включително). В кода този термин се управлява с отделен коефициент `delta_masked` и служи да фокусира натиска върху префикса, който е най-важен за асерт логиката.

- Soft distillation термин:

$$L_S = T^2 D(p_A^T \parallel p_D^T)$$

Пренася “меката” форма на AR разпределението към Diff, без да се фиксира само един токен; p^T са температурно-скалирани вероятности.

- Top-K set distillation термин:

$$L_K = -\log \sum_{v \in V_K} Q_D(v)$$

Концентрира вероятностната маса на Diff в топ-K набора на AR и подобрява шанса за асерт при decode; V_K е множеството от топ-K AR кандидати.

Важно техническо поведение в кода: при коефициент 0 съответният термин се пропуска напълно от изчислението (`alpha`, `beta`, `rho`, `chi`, `delta`, `delta_masked`, `eta`, `gamma`), което позволява експериментите да се правят с минимален излишен compute и с ясна изолация на ефекта от всеки компонент.

Механизъм на JAX компилацията: в `TiDAR/model/Run_training.py` функцията `_run_chunk` е JIT-компилирана чрез `@partial(jax.jit, static_argnames=...)`, като loss коефициентите са подадени като static аргументи. При нова комбинация от стойности се компилира нов изпълним вариант, а при повторение на същата комбинация се използва кешираният вариант. Тъй като branch-овете в `TiDAR/model/Training_step.py` са условни по коефициент, изключените термини се `prune`-ват от съответния `jaxpr`.

3.8 Резултати от greedy run-ове (135M vs 360M)

Тази секция обобщава резултати от реално тренирани TiDAR варианти с `draft_len=8`: 135M и 360M. Данните са от продължителен run (около 5 часа), при който по-големият модел е стартиран с по-голям batch, а логовете са подравнени по optimizer стъпки.

Легенда за сравняваните модели:

- 135M модел - run на RTX 5090 (32GB), платена цена за наем 5.23 USD;
- 360M модел - run на RTX PRO 6000 (96GB), платена цена за наем 11.81 USD.
- сива линия = делта 135M / 360M (1.0 означава равни стойности).

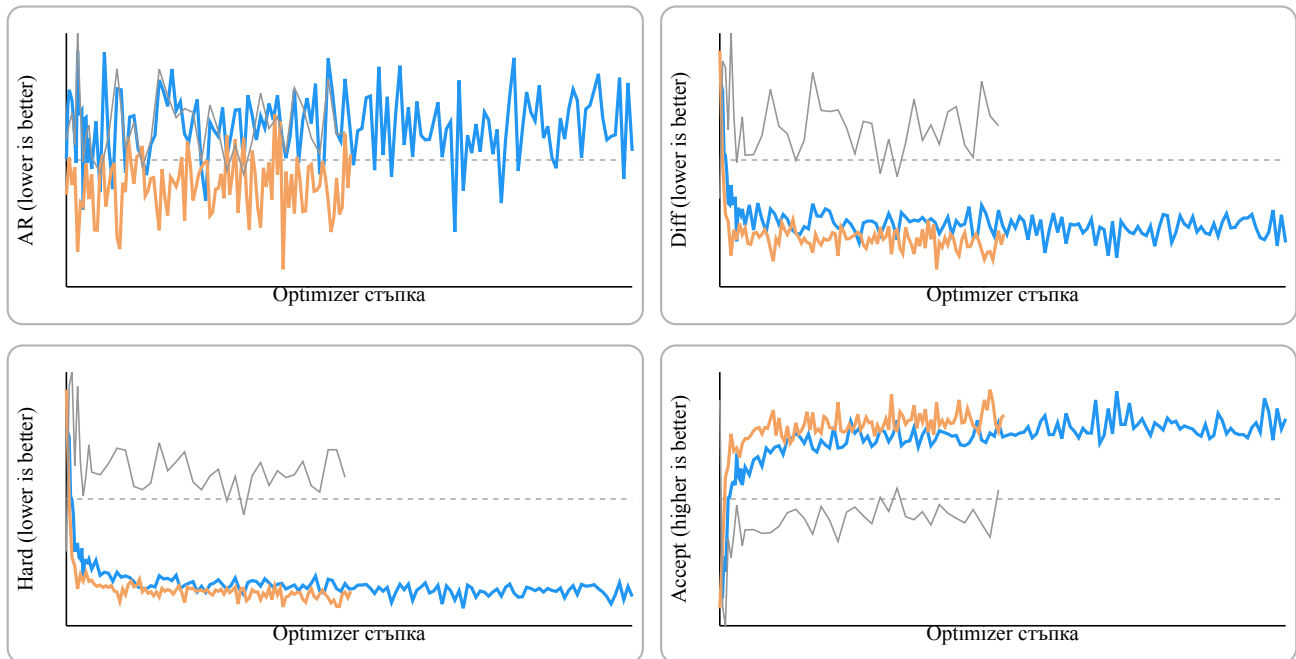


Figure 26: Обучителни метрики: 135M срещу 360M

Практически извод: 360M вариантът държи по-стабилни и по-ниски Diff/Hard загуби и по-висок greedy acceptance, но идва с осезаемо по-висока цена във време и ресурс. Моделът е приблизително 2.66 пъти по-голям по брой параметри спрямо 135M, а в реалните run-ове обучението му беше около 3-4 пъти по-бавно. Тези диаграми показват и че началните способности на базовия модел директно се отразяват върху ученето на TiDAR future prediction-ите: по-силният стартов модел учи по-стабилно тази задача, което е консистентно с по-големия му капацитет и с факта, че е предварително трениран върху повече данни. За проекта това е важен ориентир за trade-off между качество, цена и време за итерация при TiDAR training.

3.9 Допълнителен експеримент: TinyStories attention head sweep

Проведен е кратък head sweep върху TinyStories с четири конфигурации: h6 MHA (baseline), h18 MHA, h18/kv6 GQA и контролен вариант h6 с head_dim=32. Целта е бърза проверка дали промяната в attention head конфигурацията води до съществено разделяне по GIANT/TiDAR обучителните метрики.

Легенда на конфигурациите:

- h6 MHA (baseline)
- h18 MHA
- h18/kv6 GQA
- h6 head_dim=32

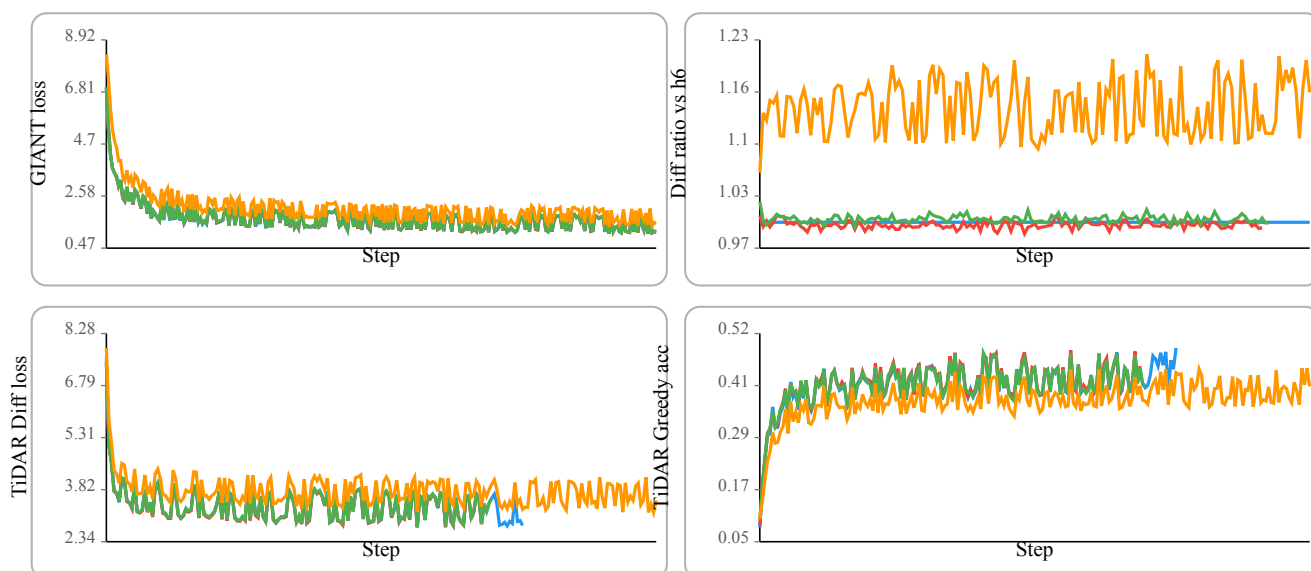


Figure 27: TinyStories head sweep: компактен преглед на 4 ключови обучителни метрики

Резултатите показват ограничено разделяне между сравнимите по ширина конфигурации (h6/h18/GQA) в рамките на краткия run прозорец. Увеличаването на броя heads само по себе си не дава ясно доминираща печалба по качество за този setup, докато GQA остава близо до baseline при умерено по-добра ефективност. Контролният h6 head_dim=32 режим е значително по-бърз, но с отчетлив спад в качеството, поради което се интерпретира като trade-off вариант, а не като пряк “по-добър” заместител.

3.10 Проведен експеримент с различни loss функции и конфигурации

В този раздел е представен контролиран експеримент за TiDAR post-training с множество loss конфигурации. Основната цел е да се оцени кои комбинации подобряват ключовите метрики (Diffusion loss и Greedy acceptance), като едновременно с това се запази качеството на оригиналния AR модел (без значимо влошаване на AR loss спрямо последните итерации от базовото GIANT обучение). Дизайнът е избран така, че да поддържа бърз експериментален цикъл: промяна на loss конфигурация, кратък run, метрика-анализ и следваща итерация.

3.10.0 Избор на TinyStories и базова конфигурация

За експерименталната серия е избран TinyStories, защото е сравнително малък корпус с ниска ентропия. Това позволява бързи итерации и ясна интерпретация на ефекта от loss промените. В практиката този тип корпус се научава стабилно и от базов модел около 30M параметъра, без да е необходимо пълно минаване през целия набор във всеки run.

В конкретната серия базовият GIANT модел (30M параметъра) е обучен върху около 1.05 милиарда токена за приблизително 33 минути на RTX 5090 (0.89 USD/час), а всеки TiDAR post-training run е отнемал около 2.5 часа на същия хардуер. Общата експериментална кампания е

проведена в рамките на една седмица с междинен анализ след всеки run (включително логитни хистограми), при приблизителен общ разход около 32 USD.

Базовият GIANT модел в тази серия е дефиниран със следната конфигурация:

```
model:
  embedding_size: 384
  num_heads: 6
  num_kv_heads: 2
  num_layers: 18
  feed_forward_size: 1024
  context_length: 512
training:
  batch_size: 32
  gradient_accumulation: 4
stages:
  - dataset: tinystories_512
    seq_len: 512
    epochs: 2
    fraction: 0.75
```

След базовото AR обучение се преминава към TiDAR post-training със същата архитектура, като началният режим е $\alpha=1$, $\beta=1$:

```
tidar:
  draft_length: 6
training:
  batch_size: 16
  gradient_accumulation: 4
loss:
  alpha: 1.0
  beta: 1.0
  rho: 0.0
  chi: 0.0
  delta: 0.0
  eta: 0.0
stages:
  - dataset: tinystories_512
```

```
seq_len: 512
epochs: 3
fraction: 0.75
```

Примерни изречения/промпт фрагменти от корпуса:

- “There was a small village.”
- “Once upon a time, a little girl...”
- “In the forest, a tiny fox...”
- “One day, the teacher said...”
- “The robot wanted to learn...”

3.10.1 Експериментален протокол и избор на checkpoint

Основният TiDAR baseline run ($\alpha=1$, $\beta=1$) достига 121566 optimizer стъпки. За сравнителната част е избран checkpoint около 90k, от който се пускат над 10 различни loss конфигурации до приблизително 121k, за да се измери ефектът им върху Diffusion/Greedy метриките при близки стартови условия.

Изборът на 90k е целенасочен: в този етап baseline режимът започва видимо да забавя нормалното си учене при $\alpha=\beta=1$, което го прави подходяща точка за branch сравнения. Част от експериментите са стартирани и от нулева стъпка за допълнителна проверка на ранната динамика.

Branch-based дизайнът е силен за сравнение, защото:

- елиминира ефекта от различна ранна инициализация;
- сравнява режимите върху близка checkpoint основа;
- позволява директни delta криви спрямо baseline по една и съща step ос.

Във файла са използвани два прозореца на анализ:

- пълен прозорец (за стабилния run) 0-121k за глобална динамика;
- zoom прозорец 90k-121k за head-to-head сравнение между branch вариантите.

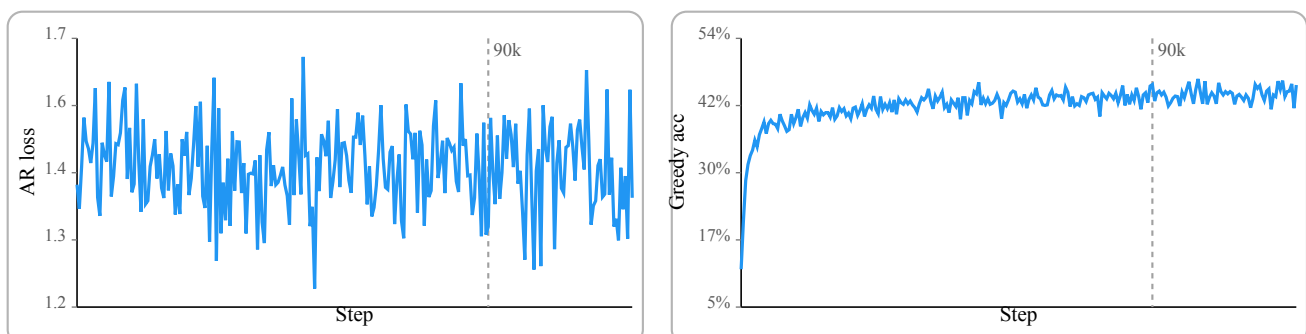


Figure 28: Базова траектория и branch cut при 90k стъпка

3.10.2 Сравнявани loss режими

Легендата по-долу обобщава сравняваните конфигурации в sweep-а. Тя покрива както “меки” alignment сигнали (KL/Distill), така и “твърди” acceptance-ориентирани сигнали (delta, delta_masked, top-k set), което е методологично правилно за TiDAR, където целта не е само нисък loss, а по-ефективен speculative decode.

Легенда на вариантите:

-  Stable - $\alpha=1$, $\beta=1$, референтна линия;
-  KL-only - $\alpha=0.2$, $\beta=0$, $\rho=0.5$, $\delta=0.3$, без Diff CE;
-  KL-keep - $\alpha=1$, $\beta=1$, $\rho=0.1$, $\delta=0.3$, умерен alignment;
-  Distill - $\alpha=1$, $\beta=1$, $\eta=0.04$, $T=2$, мека дистилация AR->Diff;
-  Greedy eta - агресивен agreement/дистилация режим;
-  Bigger beta - $\alpha=1$, $\beta=5$, усилен Diff натиск;
-  Top-K set - $\gamma=0.01$, $\gamma_{\text{topk}}=8$, късен stage;
-  Big Gamma+Delta - силен Top-K + hard agreement;
-  Masked Delta later - частичен 90k+ run с delta_masked;
-  Delta masked early - кратък ранен run под 30k.

Run-ове, които водят до твърде бърза деградация на AR quality (catastrophic forgetting) или показват незначим ефект спрямо основната група, не са включени във финалното сравнение в тази секция.

3.10.3 Интерпретация на AR и Diff кривите

AR панелите показват, че стабилният режим и умерените добавки (KL-keep, Distill) запазват близка и устойчива AR динамика в 90k-121k. Това означава, че тези варианти не разрушават основната next-token способност.

Diff панелите дават по-силен разделителен сигнал:

- при KL-only ($\beta=0$) Diff loss след 90k става нефункционален като сравним индикатор (затова е изключен от zoom Diff панела);
- KL-keep и Distill остават в сравним диапазон със stable, което е знак, че дифузионният клон продължава да учи полезно, докато се добавя alignment.

Изводът е, че пълното изключване на Diff CE може да повиши някои acceptance метрики краткосрочно, но отслабва контрола върху самата Diff реконструкция и прави режима по-рисков за дълги run-ове.

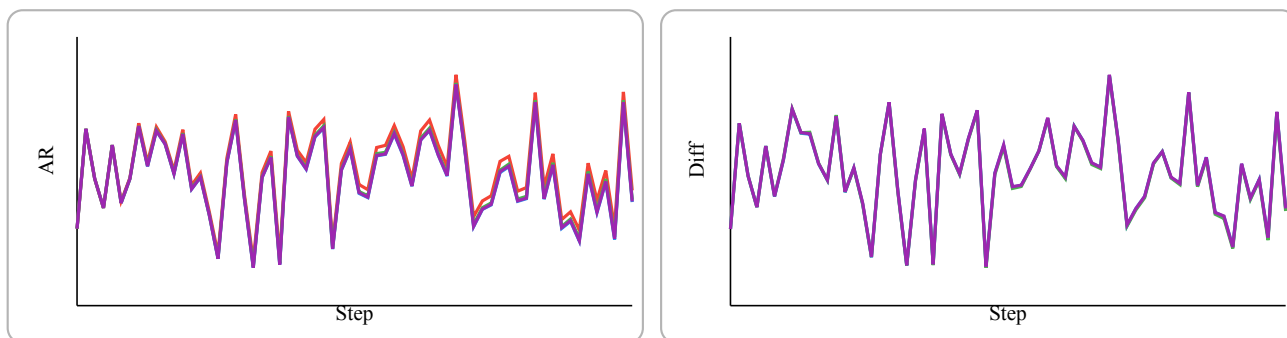


Figure 29: AR и Diff zoom сравнение за 90k-121k

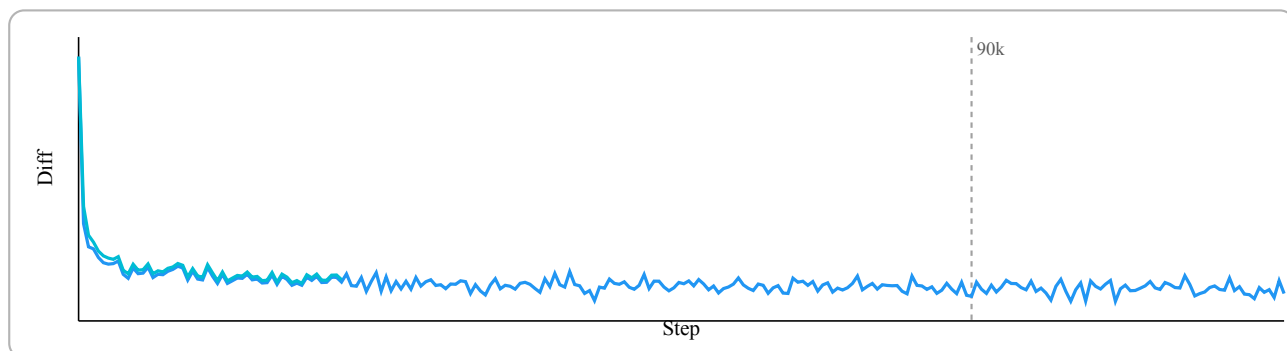


Figure 30: Пълен Diff прозорец: stable срещу ранен delta_masked run

3.10.4 Greedy acceptance: основен таргет на sweep-a

Greedy acceptance е централната метрика в документа ($\text{argmax}(\text{AR}) == \text{argmax}(\text{Diff})$ по валидни позиции). Показаните панели имат три нива на интерпретация:

- absolute панели (90k-121k): сравнение на реалните траектории;
- delta панели спрямо stable: видимост дали вариантът е устойчиво над/под baseline;
- extended панели: добавят агресивни режими и по-късни експериментални хипотези.

Най-важните наблюдения за практиката са:

- KL-keep и Distill дават по-добър баланс между acceptance печалба и запазена training стабилност;
- KL-only може да покаже моментни плюсове в greedy, но на цената на “изключен” Diff обучителен сигнал;
- по-агресивните режими (напр. Big Gamma+Delta) изискват внимателно stage планиране, защото увеличават чувствителността към хиперпараметри;
- delta_masked (късен и ранен вариант) е концептуално обещаващ, защото натиска точно acceptance-критичния префикс, но частичният характер на част от run-овете налага предпазлива интерпретация.

Част от run-овете са прекратявани по-рано, когато се наблюдава силно покачване на AR loss (catastrophic forgetting) или когато Diffusion/Greedy метриците тръгват устойчиво под основната

група. Поради това някои криви са по-къси и не достигат до последните точки от общия прозорец. Всички логове и checkpoint-и от тези run-ове са запазени в репото.

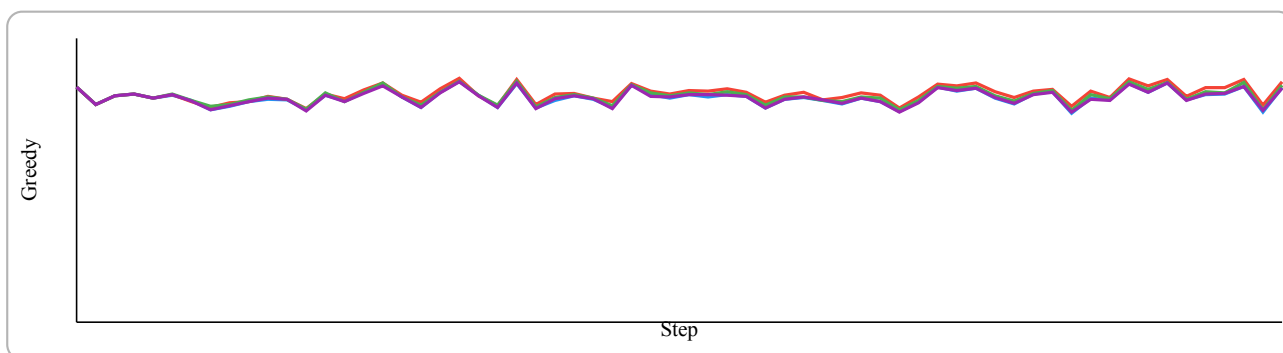


Figure 31: Greedy acceptance: core variants в 90k-121k

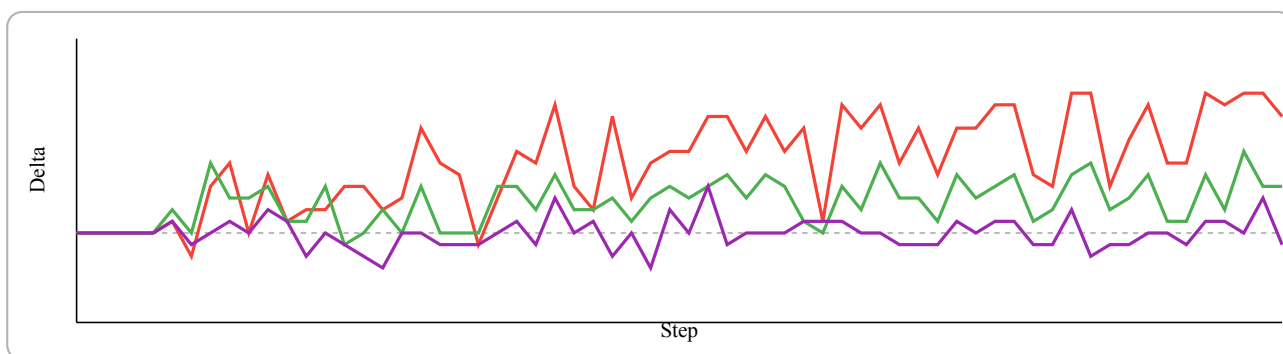


Figure 32: Delta срещу stable за core вариантите

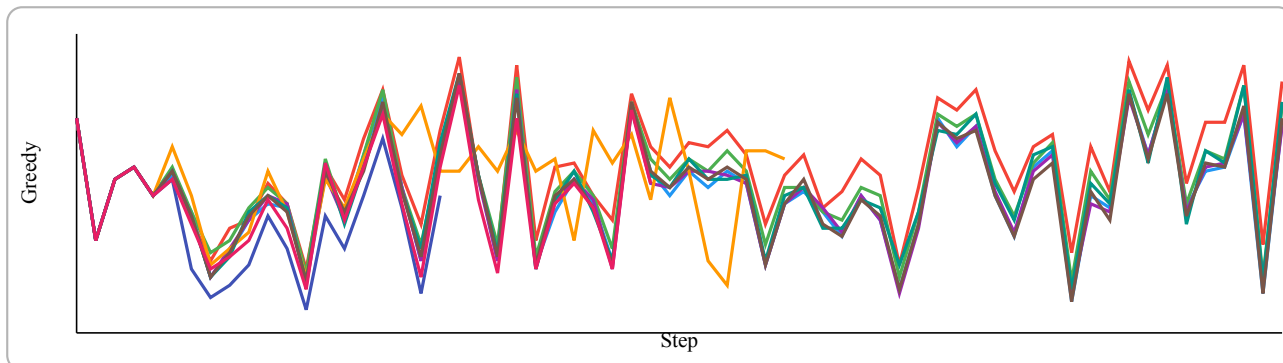


Figure 33: Greedy acceptance: разширен набор от всички варианти

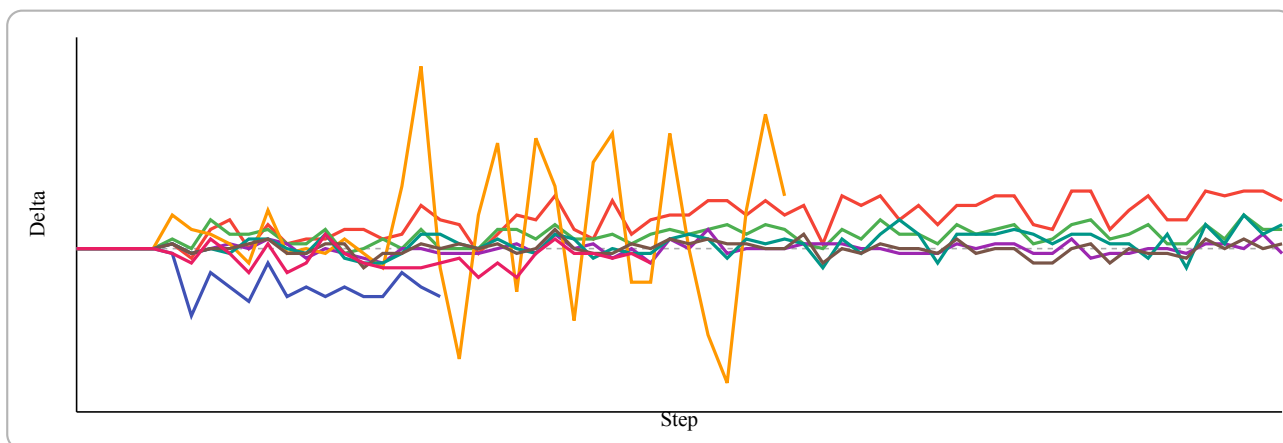


Figure 34: Delta срещу stable за разширения набор

3.10.5 Inference резултати (Accept/Iter таблица)

Секцията “Inference Results” проверява дали training ефектите се пренасят при реален greedy decode (temperature=0, draft_len=6) върху 10 промпта и три дължини на генериране (50/100/300 стъпки). Ключовите тенденции в таблицата са:

Checkpoint	Steps=50	Steps=100	Steps=300
Stable @90k	2.41 / 3.50 / 1.63	2.26 / 2.75 / 1.71	2.10 / 2.45 / 1.88
Stable @121k	2.22 / 3.27 / 1.53	2.09 / 2.68 / 1.60	2.10 / 2.69 / 1.57
KL-only @121k	2.42 / 3.27 / 1.81	2.30 / 2.68 / 1.80	2.08 / 2.49 / 1.61
KL-keep @121k	2.40 / 3.27 / 1.75	2.29 / 2.83 / 1.80	2.18 / 2.90 / 1.75
Distill @121k	2.31 / 3.50 / 1.53	2.26 / 3.09 / 1.87	2.19 / 2.74 / 1.80
SmallAR+eta @105k	2.15 / 2.88 / 1.75	2.15 / 2.75 / 1.90	2.00 / 2.43 / 1.74
Bigger beta @121.5k	2.37 / 3.27 / 1.63	2.33 / 3.09 / 1.62	1.96 / 2.27 / 1.74
Top-K set @121k	2.28 / 3.12 / 1.52	2.18 / 2.56 / 1.72	2.19 / 2.91 / 1.56
Big Gamma+Delta @105k	2.32 / 3.33 / 1.67	2.26 / 2.86 / 1.79	2.09 / 2.59 / 1.71
Masked Delta later @105k	2.30 / 3.12 / 1.72	2.34 / 2.86 / 1.72	2.01 / 2.40 / 1.67

Table 1: Accept/Iter summary върху 10 TinyStories промпта

- при Steps=300 най-силен среден резултат дават Distill (2.19) и Top-K set (2.19), следвани от KL-keep (2.18), над Stable (2.10);
- при Steps=100 Stable bigger beta достига висока средна стойност (2.33), но при дългия хоризонт (300) пада до 1.96, което подсказва по-слаба устойчивост;
- SmallAR big greedy eta и Masked Delta later са по-агресивни режими и остават под най-добрите устойчиви варианти при по-дълги генерации.

Това потвърждава работната хипотеза, че умереният alignment (KL-keep или Distill) е по-надежден за обща decode ефективност от прекалено силни наказания/насърчения върху отделен компонент.

3.10.6 Локален механистичен анализ чрез logits хистограми

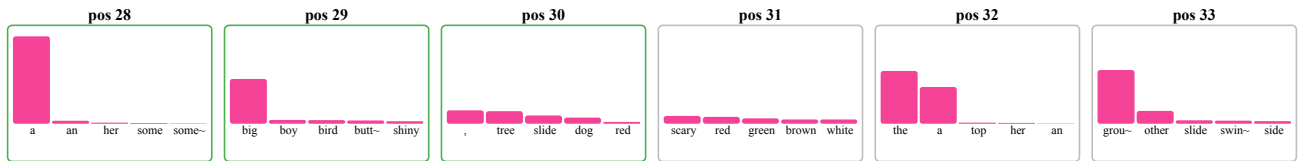
Логитните хистограми дават позиционен анализ на механизма за accept/reject и допълват агрегираните метрики с локална причинна интерпретация. Наблюдава се следният типичен модел:

- в ранните позиции на draft блока AR и Diff често съвпадат по top-1 и токените се приемат;
- при нееднозначни позиции Diff измества top-1 и започват rejection-и;
- точно тези позиции обясняват защо prefix-ориентирани loss-и (като delta_masked) имат смисъл: те таргетират момента, в който acceptance веригата се прекъсва.

Този тип визуализация е силен аргумент, че sweep-ът не е “black-box” оптимизация по една метрика, а контролирано търсене на причинно обясними подобрения.

Prompt: “Once upon” | Checkpoint: “step_0121566.npz” Target position: 30 | Block: 28-33 | Draft len: 6

AR verify logits (top-5, sorted by AR)



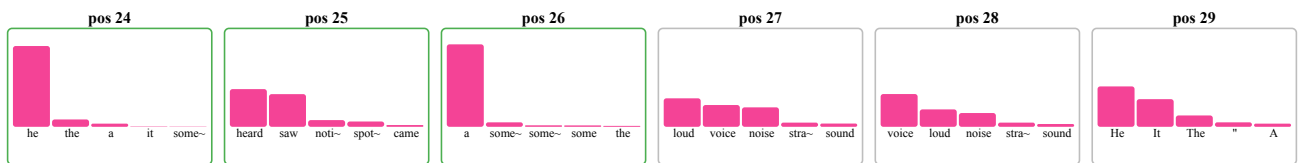
Diff draft logits (same tokens, AR order)



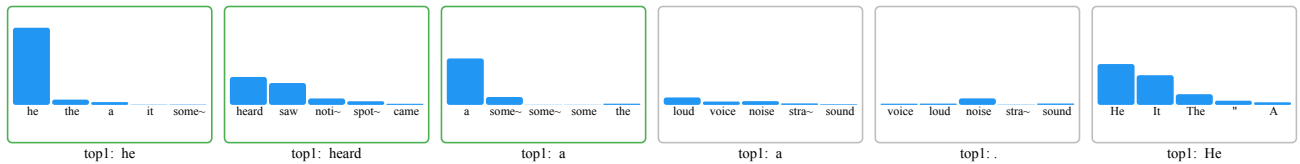
Sentence context One day, she saw a big, scary dog.

Prompt: “In the forest, a tiny fox” | Checkpoint: “step_0121566.npz” Target position: 27 | Block: 24-29 | Draft len: 6

AR verify logits (top-5, sorted by AR)



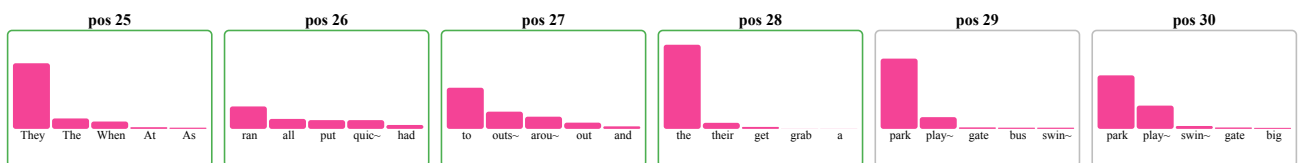
Diff draft logits (same tokens, AR order)



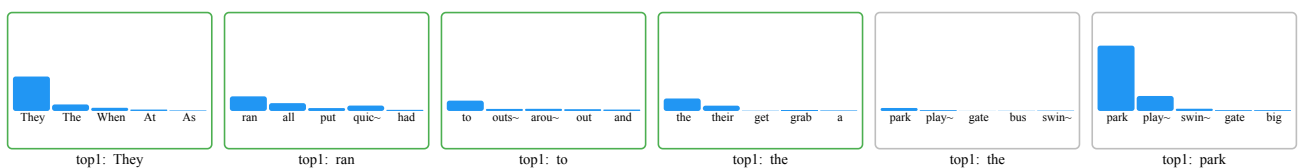
Sentence context Suddenly, he heard a loud noise.

Prompt: “One day, the teacher said” | Checkpoint: “step_0121566.npz” Target position: 29 | Block: 25-30 | Draft len: 6

AR verify logits (top-5, sorted by AR)



Diff draft logits (same tokens, AR order)



Sentence context They ran to the park and saw a big slide.

3.10.7 Финален на експеримента и план за продължение

Проведеният експеримент изпълнява основната си цел: да валидира end-to-end TiDAR training/inference pipeline и да даде първа сравнителна картина за ефекта от различни loss конфигурации. В същото време резултатите показват, че при текущата постановка разделителната способност между вариантите е ограничена.

Основното наблюдение е силното припокриване на кривите между различните директории/конфигурации. В значителна част от диапазона разликите са минимални и на места практически неразличими визуално. Това е индикация, че в текущия режим доминира влиянието на корпуса, а не на конкретния избор на auxiliary loss.

Работната интерпретация е, че TinyStories е нискоентропиен и силно структуриран корпус. При такава среда локална грешка в отделна позиция често не води до трайно разминаване по следващите токени, защото контекстът остава лесен за възстановяване. Поради това режими като delta_masked могат да изглеждат по-слаби в ранния диапазон (0-30k), без това задължително да означава по-нисък потенциал на самия метод в по-труден домейн.

Втори ключов фактор е мащабната разлика спрямо оригиналната експериментална среда на NVIDIA. Тук базовият модел и тренировъчния корпус са приблизително 50 пъти по-малки, докато референтната постановка използва значително по-голям предварително обучен модел (Qwen 1.5B) и много по-широка предтренировъчна база. Това ограничение директно намалява чувствителността на експеримента към фини ефекти от loss комбинациите.

Планирано продължение до защитата (следващи 3 месеца):

- втори експериментален цикъл върху по-информативен корпус от типа OpenWebText/Wikipedia/куриран web text;
- увеличение на модела и тренировъчния обем, така че loss режимите да се разграничат по-ясно;
- избор на най-устойчивите конфигурации от този междинен етап и пренасяне към финален training диапазон от порядъка 300M-1B параметри и 5B-30B токена.

Следователно текущият експеримент се приема като междинен, но необходим етап: системата е валидирана, наблюдавани са ограниченията на малък/лесен корпус, и е дефиниран конкретен план за доразвитие преди финалната защита.

ЧЕТВЪРТА ГЛАВА

РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ

4.1 Инсталация

Базовият начин за стартиране е чрез Docker образите в CICD/Docker/.

Минимален сценарий:

```
docker run --pull always -d --gpus all \
  --name giant-training \
  --mount type=bind,source="$HOME/GIANT",target=/proj \
  bonanc/giant-training:latest
```

```
docker exec -it giant-training bash
```

За automatic-S3-sync workflow е наличен отделен образ CICD/Docker/giant-training-S3/, за който е нужна база като Minio или подобна в облака, за постоянно обновяване на данните. При слаб интернет от страна на developer машината това е не препоръчително. S3 tool-овете са достатъчни за базовия вариант.

4.2 Стартиране на средата и създаване на експеримент

Под “проект” в тази система се разбира комбинация от:

- глобална конфигурация (GIANT/v2/Global_Config.yml, TiDAR/Global_Config.yml);
- локална model/training конфигурация (Config.yml);
- директории за dataset/checkpoints/logs.

Първата стъпка е избор на data root и checkpoint root, след което се изпълняват pipeline и training скриптовете.

4.3 Подготовка и импорт на datasets

Импортиране на datasets.

Примери:

```
python GIANT/v2/data_pipeline/build_corpus.py \
  --config GIANT/v2/data_pipeline/Config.yml
python TiDAR/data_pipeline/Run_pipeline.py \
  --config TiDAR/data_pipeline/data_configs/Greedy_exp_500m.yml
```

Системата ще генерира Arrow shard-ове и manifest/statistics файлове.

4.4 Работа с training stages

Stage структурата играе ролята на curriculum по време на обучение.

4.4.1 Добавяне и пренареждане на stages

Еквивалент: добавяне и пренареждане на stages в YAML. Всеки stage може да задава seq_len, epochs, fraction и stage-local loss коефициенти.

4.4.2 Избор на checkpoint

Еквивалент: избор на конкретна checkpoint версия за инференс.

```
python GIANT/v2/model/Generate_faster.py \  
  --checkpoint latest  
python TiDAR/model/inference.py \  
  --checkpoint /proj/giant-data/TiDAR/checkpoints/params/step_0012100.npz
```

4.4.3 Разделяне на run на етапи

Еквивалент: разделяне на training run на етапи с различни objectives или подмяна на config по време на следващ stage.

4.4.4 Нулиране на експеримент

Еквивалент: reset на конкретен експеримент чрез нов checkpoint root или презаписване на stage output директория (pipeline регенерация).

4.5 Управление на изпълнението

Основни команди за runtime управление:

- старт на обучение: Run_training.py;
- пауза/спиране: SIGINT/SIGTERM (graceful stop след текущ chunk);
- resume: --resume latest или път до конкретна checkpoint;
- inference режими: greedy (--temperature 0) или sampling (--temperature > 0, --top_k).

4.6 Контрол на KV cache и inference мащаб

За инференс производителност Generate_faster.py поддържа KV cache bucket избор:

- автоматичен избор;
- конфигурационни buckets;
- CLI override чрез --kv_cache_buckets;
- изключване чрез --disable_kv_buckets.

Това позволява баланс между памет и latency, особено при различни prompt/steps комбинации.

4.7 Настройка на training параметри

Настройка на hyperparameters и loss коефициенти.

За TiDAR основните контроли са в training.loss:

- alpha, beta;
- rho, chi;
- delta, delta_masked;
- eta, eta_T;
- gamma, gamma_topk.

4.8 Клавишни комбинации и бързи команди

Работният процес е CLI-first. Типични бързи команди:

- `python GIANT/v2/model/Run_training.py --resume latest`
- `python GIANT/v2/model/Evaluate.py --checkpoint latest`
- `python GIANT/v2/model/Chat.py --chat --steps 128`
- `python GIANT/v2/model/Generate_faster.py --prompt "Hello" --steps 64 --verbose`
- `python TiDAR/model/Run_training.py --config TiDAR/model/Config_135m.yml`
- `python TiDAR/model/inference.py --prompt "Hello" --draft_len 8 --temperature 0.0`

Главният контейнер е базова runtime среда за обучение и инференс, която съдържа само необходимите системни и ML зависимости: CUDA runtime/драйверно-съвместими библиотеки, JAX/Flax/Optax/Orbach стек, Python tooling и build инструменти за възпроизводимо изпълнение на скриптовете. Така се гарантира, че training и inference поведението е консистентно между локална машина и remote GPU среда.

Репото и dataset/checkpoint данните не са вградени в самия Docker образ. Те се добавят след стартиране на контейнера чрез bind mount, sync или S3 workflow. Този подход държи образа по-малък, ускорява обновяването на кода и данните, и избягва ненужно преизграждане на image при всяка промяна в експериментите.

ЗАКЛЮЧЕНИЕ

В настоящата дипломна работа е реализирана цялостна инженерна система за разработка на езиков модел, която обединява data pipeline, обучение, инференс и експериментална инфраструктура в единна codebase.

Основните постижения могат да се обобщят така:

- проектиран е стабилен pipeline за подготовка на данни с shard-ване, manifest и статистика;
- реализирано е обучение на decoder-only Transformer модел с възможност за resume и възстановяване на състояние;
- реализиран е ускорен KV-cache инференс с practically useful CLI сценарии;
- изградена е инфраструктура за remote GPU execution, Docker deployment и S3 синхронизация;
- реализирано е съществено TiDAR разширение с Anchor-TiDAR decode и разширен набор от loss термини за контрол върху acceptance/quality поведението.

Работата показва, че архитектурното разделяне между базова платформа (GIANT/v2) и изследователско разширение (TiDAR) е удачно: базовата част остава стабилна и преизползваема, а експерименталната част може да се развива итеративно без разрушаване на основния workflow.

Възможности за бъдещо развитие:

- добавяне на multi-GPU distributed обучение;
- интеграция на допълнителни архитектурни оптимизации (например MoE/MLA);
- по-богат automated evaluation пакет за quality/speed trade-off;
- допълнително формализиране на експерименталните протоколи и автоматично генериране на отчети.

Използвана литература

- [1] Vaswani, A. et al. **Attention Is All You Need**. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- Ba, J. L., Kiros, J. R., Hinton, G. E. **Layer Normalization**. 2016. <https://arxiv.org/abs/1607.06450>
- [10] Zhang, B., Sennrich, R. **Root Mean Square Layer Normalization (RMSNorm)**. 2019. <https://arxiv.org/abs/1910.07467>
- Srivastava, N. et al. **Dropout: A Simple Way to Prevent Neural Networks from Overfitting**. 2014. <https://jmlr.org/papers/v15/srivastava14a.html>
- [11] Su, J. et al. **RoFormer: Enhanced Transformer with Rotary Position Embedding**. 2021. <https://arxiv.org/abs/2104.09864>
- [12] Shazeer, N. **GLU Variants Improve Transformer**. 2020. <https://arxiv.org/abs/2002.05202>
- [6] Kingma, D. P., Ba, J. **Adam: A Method for Stochastic Optimization**. 2014. <https://arxiv.org/abs/1412.6980>
- Loshchilov, I., Hutter, F. **Decoupled Weight Decay Regularization (AdamW)**. 2017. <https://arxiv.org/abs/1711.05101>
- Dao, T. et al. **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. 2022. <https://arxiv.org/abs/2205.14135>
- Dao, T. **FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning**. 2023. <https://arxiv.org/abs/2307.08691>
- Leviathan, Y., Kalman, M., Matias, Y. **Fast Inference from Transformers via Speculative Decoding**. 2023. <https://arxiv.org/abs/2211.17192>
- Stern, M. et al. **Blockwise Parallel Decoding for Deep Autoregressive Models**. 2018. <https://arxiv.org/abs/1811.03115>
- Cai, T. et al. **Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads**. 2024. <https://arxiv.org/abs/2401.10774>
- Li, Y. et al. **EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty**. 2024. <https://arxiv.org/abs/2401.15077>
- NVIDIA Research. **Think in Diffusion, Talk in Autoregression (TiDAR)**. 2025. <https://arxiv.org/pdf/2511.08923>
- [8] Touvron, H. et al. **LLaMA: Open and Efficient Foundation Language Models**. 2023. <https://arxiv.org/abs/2302.13971>

- [5] Hoffmann, J. et al. **Training Compute-Optimal Large Language Models (Chinchilla)**. 2022. <https://arxiv.org/abs/2203.15556>
- Kaplan, J. et al. **Scaling Laws for Neural Language Models**. 2020. <https://arxiv.org/abs/2001.08361>
- [2] Kwon, W. et al. **Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM)**. 2023. <https://arxiv.org/abs/2309.06180>
- NVIDIA. **NVIDIA Ampere Architecture In-Depth**. 2020. <https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/>
- NVIDIA. **NVIDIA A100 Tensor Core GPU Architecture Whitepaper**. 2020. <https://resources.nvidia.com/en-us-tensor-core/nvidia-ampere-architecture-whitepaper>
- NVIDIA. **NVIDIA A40 Datasheet**. 2020. <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/documents/nvidia-rtx-a40-datasheet.pdf>
- JAX Documentation. <https://docs.jax.dev/>
- Flax Documentation. <https://flax.readthedocs.io/>
- Optax Documentation. <https://optax.readthedocs.io/>
- [4] Orbx Checkpointing Documentation. <https://orbx.readthedocs.io/>
- [3] Hugging Face Datasets Documentation. <https://huggingface.co/docs/datasets>
- Hugging Face Transformers Documentation. <https://huggingface.co/docs/transformers>
- Docker Documentation. <https://docs.docker.com/>
- Tailscale Documentation. <https://tailscale.com/kb>
- s5cmd Documentation. <https://github.com/peak/s5cmd>
- [9] Andrej Karpathy. **Let's build GPT: from scratch, in code, spelled out**. YouTube, 2023. <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- Andrej Karpathy. **Neural Networks: Zero to Hero** (video playlist). YouTube. <https://karpathy.ai/zero-to-hero.html>
- Andrej Karpathy. **Intro to Large Language Models**. YouTube, 2023. https://www.youtube.com/watch?v=zjkBMFhNj_g
- Typst Documentation. <https://typst.app/docs/>
- [7] Brutlag, J. **Speed Matters for Google Web Search**. Google, 2009. https://services.google.com/fh/files/blogs/google_delayexp.pdf
- DeepSeek-AI. **DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model (MLA + MoE)**, 2024. <https://arxiv.org/abs/2405.04434>

- DeepSeek-AI. **DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models**, 2024. <https://arxiv.org/abs/2401.06066>
- Beltagy, I., Peters, M. E., Cohan, A. **Longformer: The Long-Document Transformer** (sliding-window attention), 2020. <https://arxiv.org/abs/2004.05150>

СЪДЪРЖАНИЕ

ПЪРВА ГЛАВА	3
ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ СИСТЕМИ, ПОДОБНИ НА GIANT	3
1.1 Базов контекст: какво е езиков модел	3
1.2 Термини (кратки дефиниции)	3
1.3 Сходни технологии и инженерни практики	4
1.3.1 LLaMA семейство (Meta)	4
1.3.2 SmolLM / компактни open модели	4
1.3.3 nanoGPT / minGPT	5
1.3.4 vLLM и TensorRT-LLM	5
1.3.5 Защо скоростта е критична за LLM приложения	5
ВТОРА ГЛАВА	6
ФУНКЦИОНАЛНИ ИЗИСКВАНИЯ И АРГУМЕНТАЦИЯ НА ИЗБОРА НА СРЕДСТВА	
ЗА РАЗРАБОТКА	6
2.1 Функционални изисквания	6
2.1.1 Входни източници и ingest pipeline	6
2.1.2 Подготовка на корпус, токенизация и stage-и	6
2.1.3 Конфигурационно управление на експерименти	7
2.1.4 Мониторинг и метрики по време на изпълнение	7
2.1.5 Управление на хиперпараметри и decode политика	7
2.1.6 Артефакти, checkpoint-и и отчетност	7
2.2 Аргументация за избор на езика и софтуерните средства	8
2.2.1 Избор на езика за програмиране	8
2.2.2 Избор на платформа за разработка	8
2.2.3 Избор на изчислителен фреймуърк	8
2.2.4 Избор на optimizer/checkpoint инструменти	9
2.2.5 Избор на средства за deployment и синхронизация	9
2.3 Структура на данните и вътрешен модел на системата	9
2.3.1 Формат на данните и логика на съхранение	9
2.3.2 Stage конфигурация и curriculum модел	9
2.3.3 Конфигурационни домейни на системата	10

2.3.4	Схема на dataset записите в shard	10
2.3.5	Източници и ingest адаптери	10
2.4	Цялостна архитектура и структурна организация	10
2.4.1	Интерфейсен слой (CLI)	10
2.4.2	Управляващи модули	11
2.4.3	Основни entry points	11
2.5	Основен математически модел на трансформер архитектурата	12
2.5.1	Теоретична отправна точка: Attention Is All You Need	12
2.5.2	Multi-head attention: защо са нужни “много глави”	14
2.5.3	От класически Transformer към decoder-only LLM	15
2.5.4	Нормализация, активации и позиционна информация в GIANT	17
2.5.5	Обучителна цел (AR) и връзка с TiDAR	19
2.6	Спекулативно декодиране (въведение преди TiDAR)	20
2.6.1	Защо single-user decode не използва добре GPU	21
ТРЕТА ГЛАВА		23
РЕАЛИЗАЦИЯ НА ПРИЛОЖЕНИЕТО		23
3.1	Реализация на потребителски интерфейс	23
3.1.1	Основни входни точки (CLI)	23
3.1.2	Наблюдение и визуализация на резултати	23
3.1.3	Управление на stage прогреса	23
3.1.4	Управление на параметри	23
3.1.5	Добавяне на входни корпуси	23
3.1.6	Експорт на артефакти	24
3.1.7	Примерен training run в терминал	24
3.2	Реализация на основните системни модули	26
3.2.1	ConfigLoaderManager	26
3.2.2	TrainLoopOrchestrator	26
3.2.3	DataLoaderManager	28
3.2.4	CheckpointManager	29
3.2.5	InferenceManager	29
3.2.6	RemoteOpsManager	33

3.3 Реализация на data pipeline	34
3.3.1 Документ като източник на токени	34
3.3.2 Преобразуване от документ към sequence прозорци	36
3.3.3 Директно четене и минимизиране на IO overhead	36
3.3.4 Шардване	36
3.3.5 Loader batch изграждане	36
3.3.6 Допълнителни preprocessing опции	36
3.4 Имплементация на “Think in Diffusion, Talk in AutoRegression”	37
3.4.0.1 Контекст: от класически speculative decode към TiDAR	37
3.4.0.2 Основна идея на TiDAR в един forward pass	38
3.4.0.3 Структурирани маски и достъп по позиции	38
3.4.0.4 Prefill маска	39
3.4.0.5 Training маска	40
3.4.0.6 Loss формулировка и баланс AR:Diff	41
3.4.0.7 Anchor-TiDAR (финален акцент)	41
3.4.1 Представяне на dual sequence вход	42
3.4.2 Преобразуване и маскиране по позиции	42
3.4.3 KV-cache pointer commit/rollback	43
3.4.4 Извличане на verify и predraft логити	43
3.4.5 Decode сценарий (Anchor-TiDAR)	44
3.4.6 Допълнителна визуална интерпретация за Mij	44
3.5 Експеримент: Free Token Slots	45
3.5.1 Native decode attention	45
3.5.2 Kernel terms (контролен анализ)	46
3.5.3 MLP scaling	46
3.5.4 Sampling path (greedy и non-greedy)	47
3.6 Резултати: TiDAR срещу AR (A40 и H100)	48
3.7 Разширена loss функция за TiDAR	48
3.7.1 Формулировка с независими коефициенти	48
3.8 Резултати от greedy run-ове (135M vs 360M)	50
3.9 Допълнителен експеримент: TinyStories attention head sweep	51

3.10 Проведен експеримент с различни loss функции и конфигурации	52
3.10.0 Избор на TinyStories и базова конфигурация	52
3.10.1 Експериментален протокол и избор на checkpoint	54
3.10.2 Сравнявани loss режими	55
3.10.3 Интерпретация на AR и Diff кривите	55
3.10.4 Greedy acceptance: основен таргет на sweep-a	56
3.10.5 Inference резултати (Accept/Iter таблица)	58
3.10.6 Локален механистичен анализ чрез logits хистограми	58
3.10.7 Финален на експеримента и план за продължение	61
ЧЕТВЪРТА ГЛАВА	62
РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ	62
4.1 Инсталация	62
4.2 Стартиране на средата и създаване на експеримент	62
4.3 Подготовка и импорт на datasets	62
4.4 Работа с training stages	63
4.4.1 Добавяне и пренареждане на stages	63
4.4.2 Избор на checkpoint	63
4.4.3 Разделяне на run на етапи	63
4.4.4 Нулиране на експеримент	63
4.5 Управление на изпълнението	63
4.6 Контрол на KV cache и inference мащаб	63
4.7 Настройка на training параметри	63
4.8 Клавишни комбинации и бързи команди	64
ЗАКЛЮЧЕНИЕ	65
Използвана литература	66